Automated Entailment Proving in Separation Logic

PhD Candidate: Ta Quang Trung

Supervisors: Assoc. Prof. Khoo Siau Cheng
Assoc. Prof. Chin Wei Ngan

National University of Singapore – May 11th, 2018

Where I am from

Where I have been in recent 6 years

Before starting, I would like to thank...

- My supervisors, Prof. Khoo Siau Cheng and Prof. Chin Wei Ngan, for the enthusiastic guidance.
- My thesis examiners, Prof. Joxan Jaffar, Prof. David Rosenblum, Prof. Alwen Tiu, for the constructive reviews.
- Prof. Damith Rajapakse for chairing my PhD defense.
- NUS, SoC, and their professors and staffs, my family, my teachers, my friends, for all the supports and sharing.
- And all of you, for being here today.

- Example: a program that concatenates two linked lists.

```
//a recursive data structure
typedef struct Node {
  struct Node* next;
} node;

// linked list concatenation
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

- Example: a program that concatenates two linked lists.

```
//a recursive data structure
typedef struct Node {
  struct Node* next;
} node;

// linked list concatenation
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

- How to specify desired behaviours of the program `concat`?

Formal Verification of Software

- Example: a program that concatenates two linked lists.

```
//a recursive data structure
typedef struct Node {
  struct Node* next;
} node;

// linked list concatenation
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

- How to specify desired behaviours of the program `concat`?
- How to verify that `concat` is implemented correctly?

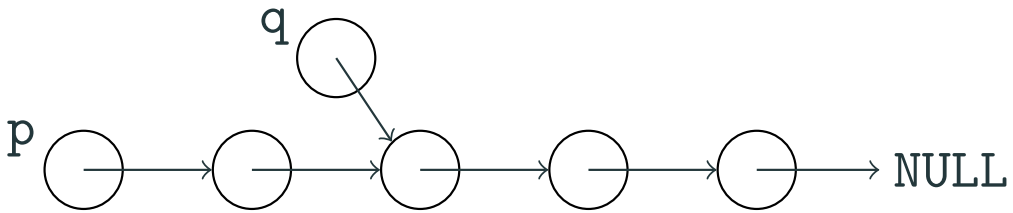
Linked List Concatenation: Specification

```
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

Linked List Concatenation: Specification

```
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

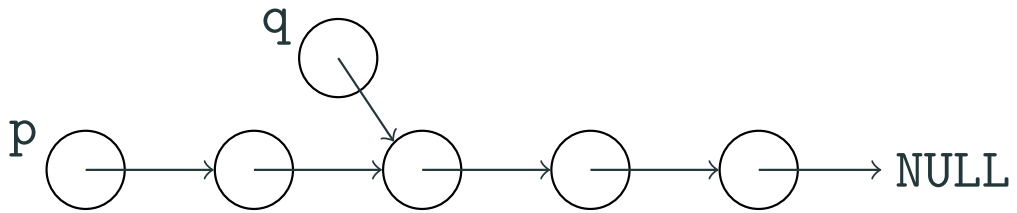
▷ Undesired input: p and q point to overlapping memory portions.



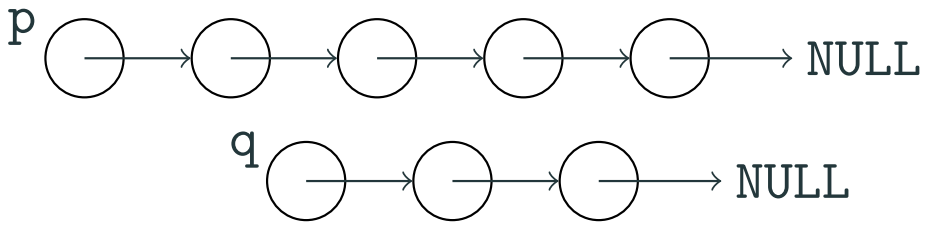
Linked List Concatenation: Specification

```
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

- Undesired input: p and q point to overlapping memory portions.



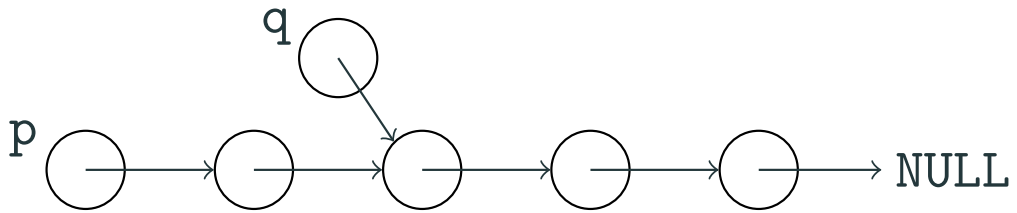
- Expected input: p and q point to separate memory portions.



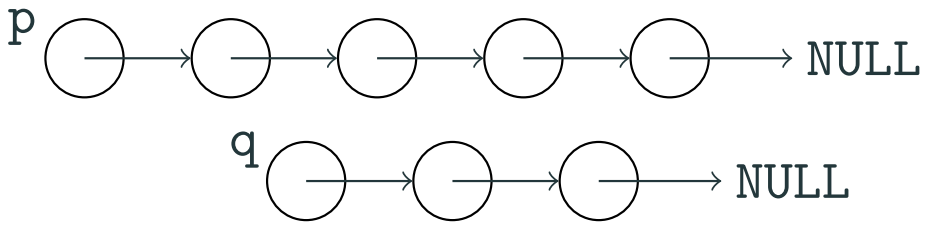
Linked List Concatenation: Specification

```
node* concat(node* p, node* q) {
  if (p == NULL) return q;
  else {
     p->next = concat(p->next, q);
    return p; }}
```

- Undesired input: p and q point to overlapping memory portions.



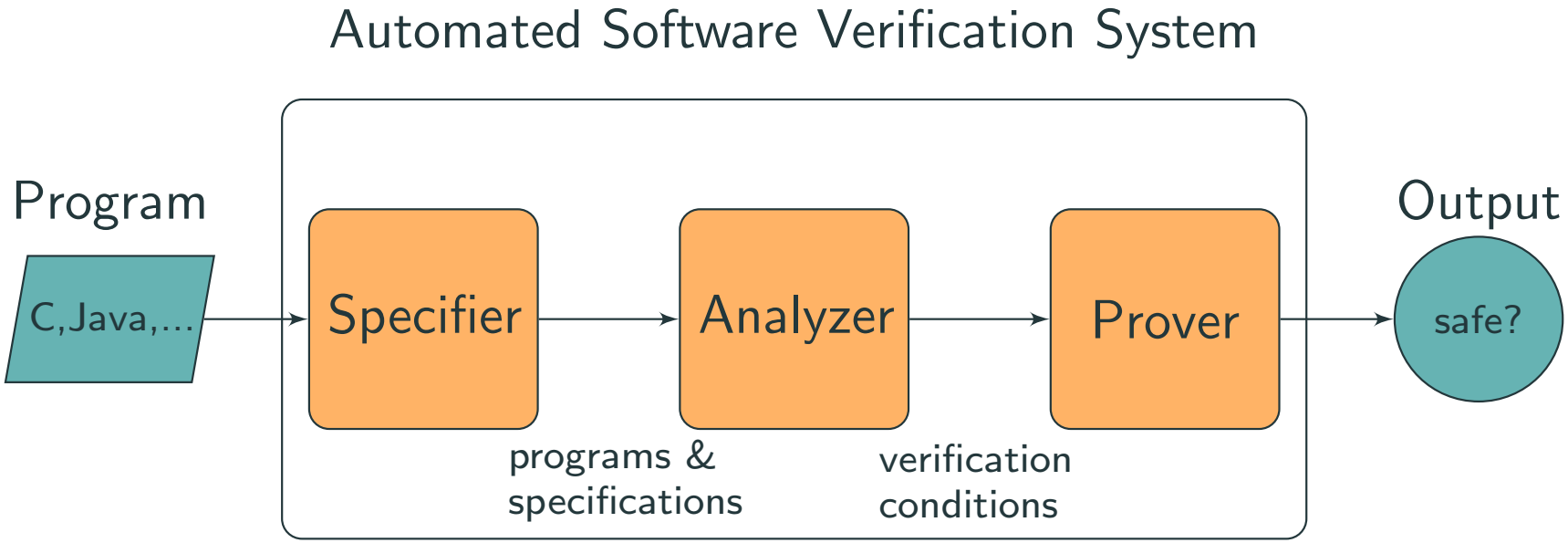
- Expected input: p and q point to separate memory portions.



Separation logic formula:

$\mathsf{list}(p, n) * \mathsf{list}(q, m)$

Software Verification Using Separation Logic



1. Specify the program behaviors using separation logic formulas.
2. Analyze the program against its specification.
3. Prove the verification conditions to determine the program's safety.

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q                    [i] Prove post-condition
                                list(p, n) * list(q, m) ∧ p=NULL ∧ res=q ⊢ list(res, n+m)
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL                    [ii] Prove pre-condition
                list(x, n−1) * list(q, m) ⊢ ∃n′, m′.(list(x, n′) * list(q, m′))
    node* y = concat(x, q);
                list(res′, n+m−1)                                  [iii] Utilize post-condition
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p                       [iv] Prove post-condition
                    p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res, n + m)
  }}
```

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q                [i] Prove post-condition
                        list(p, n) * list(q, m) ∧ p=NULL ∧ res=q ⊢ list(res, n+m)
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL                [ii] Prove pre-condition
              list(x, n−1) * list(q, m) ⊢ ∃n′, m′.(list(x, n′) * list(q, m′))
    node* y = concat(x, q);
              list(res′, n+m−1)                               [iii] Utilize post-condition
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p                   [iv] Prove post-condition
                  p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res, n + m)
  }}
```

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p,n) * list(q,m)
//@ ensures list(res, n+m)
{
  // list(p,n) * list(q,m)
  if (p == NULL)
    // list(p,n) * list(q,m) ∧ p=NULL
    return q;
    // list(p,n) * list(q,m) ∧ p=NULL ∧ res=q                    [i] Prove post-condition
                        list(p,n) * list(q,m) ∧ p=NULL ∧ res=q ⊢ list(res,n+m)
  else {
    // list(p,n) * list(q,m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x,n−1) * list(q,m) ∧ p≠NULL                    [ii] Prove pre-condition
                list(x,n−1) * list(q,m) ⊢ ∃n′,m′.(list(x,n′) * list(q,m′))
    node* y = concat(x, q);
                list(res′,n+m−1)                                 [iii] Utilize post-condition
    // p↦x * list(y,n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y,n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p                      [iv] Prove post-condition
                    p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res,n+m)
  }}
```

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p,n) * list(q,m)
//@ ensures list(res, n+m)
{
  // list(p,n) * list(q,m)
  if (p == NULL)
    // list(p,n) * list(q,m) ∧ p=NULL
    return q;
    // list(p,n) * list(q,m) ∧ p=NULL ∧ res=q                    [i] Prove post-condition
                        list(p,n) * list(q,m) ∧ p=NULL ∧ res=q ⊢ list(res,n+m)
  else {
    // list(p,n) * list(q,m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x,n−1) * list(q,m) ∧ p≠NULL                    [ii] Prove pre-condition
              list(x,n−1) * list(q,m) ⊢ ∃n',m'.(list(x,n') * list(q,m'))
    node* y = concat(x, q);
              list(res',n+m−1)                                  [iii] Utilize post-condition
    // p↦x * list(y,n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y,n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p                      [iv] Prove post-condition
                  p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res,n+m)
  }}
```

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

[i] Prove post-condition

$\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$

[ii] Prove pre-condition

$\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$

[iii] Utilize post-condition

$\mathsf{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

[i] Prove post-condition

$\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$

[ii] Prove pre-condition

$\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$

[iii] Utilize post-condition

$\mathsf{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p,n) * list(q,m)
//@ ensures list(res, n+m)
{
  // list(p,n) * list(q,m)
  if (p == NULL)
    // list(p,n) * list(q,m) ∧ p=NULL
    return q;
    // list(p,n) * list(q,m) ∧ p=NULL ∧ res=q
    list(p,n) * list(q,m) ∧ p=NULL ∧ res=q ⊢ list(res,n+m)
  else {
    // list(p,n) * list(q,m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x,n−1) * list(q,m) ∧ p≠NULL
    list(x,n−1) * list(q,m) ⊢ ∃n′,m′.(list(x,n′) * list(q,m′))
    node* y = concat(x, q);
    list(res′,n+m−1)
    // p↦x * list(y,n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y,n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p
    p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res,n+m)
  }}
```

[i] Prove post-condition

[ii] Prove pre-condition

[iii] Utilize post-condition

[iv] Prove post-condition

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p,n) * list(q,m)
//@ ensures list(res, n+m)
{
  // list(p,n) * list(q,m)
  if (p == NULL)
    // list(p,n) * list(q,m) ∧ p=NULL
    return q;
    // list(p,n) * list(q,m) ∧ p=NULL ∧ res=q
                                        [i] Prove post-condition
                        list(p,n) * list(q,m) ∧ p=NULL ∧ res=q ⊢ list(res,n+m)
  else {
    // list(p,n) * list(q,m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x,n−1) * list(q,m) ∧ p≠NULL
                                        [ii] Prove pre-condition
                list(x,n−1) * list(q,m) ⊢ ∃n′,m′.(list(x,n′) * list(q,m′))
    node* y = concat(x, q);
                list(res′,n+m−1)        [iii] Utilize post-condition
    // p↦x * list(y,n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y,n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p
                                        [iv] Prove post-condition
                        p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res,n+m)
  }}
```

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

[i] Prove post-condition

$\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$

[ii] Prove pre-condition

$\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$

$\mathsf{list}(res', n{+}m{-}1)$ [iii] Utilize post-condition

[iv] Prove post-condition

$p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

[i] Prove post-condition

$\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$

[ii] Prove pre-condition

$\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$

[iii] Utilize post-condition

$\mathsf{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \;\vee\; \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

[i] Prove post-condition

$\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$

[ii] Prove pre-condition

$\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$

[iii] Utilize post-condition

$\mathsf{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

Linked List Concatenation: Verification

$\mathsf{list}(u, n) \triangleq \mathsf{emp} \wedge u{=}\mathtt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \mathsf{list}(v, n{-}1))$

```
node* concat(node* p, node* q)
//@ requires list(p,n) * list(q,m)
//@ ensures list(res, n+m)
{
  // list(p,n) * list(q,m)
  if (p == NULL)
    // list(p,n) * list(q,m) ∧ p=NULL
    return q;
    // list(p,n) * list(q,m) ∧ p=NULL ∧ res=q                    [i] Prove post-condition
                        list(p,n) * list(q,m) ∧ p=NULL ∧ res=q ⊢ list(res,n+m)
  else {
    // list(p,n) * list(q,m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x,n−1) * list(q,m) ∧ p≠NULL                    [ii] Prove pre-condition
                list(x,n−1) * list(q,m) ⊢ ∃n′,m′.(list(x,n′) * list(q,m′))
    node* y = concat(x, q);
                list(res′,n+m−1)                                 [iii] Utilize post-condition
    // p↦x * list(y,n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y,n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p                      [iv] Prove post-condition
                    p↦y * list(y,n+m−1) ∧ p≠NULL ∧ res=p ⊢ list(res,n+m)
  }}
```

Linked List Concatenation: Verification

```
node* concat(node* p, node* q)
//@ requires list(p, n) * list(q, m)
//@ ensures list(res, n + m)
{
  // list(p, n) * list(q, m)
  if (p == NULL)
    // list(p, n) * list(q, m) ∧ p=NULL
    return q;
    // list(p, n) * list(q, m) ∧ p=NULL ∧ res=q
  else {
    // list(p, n) * list(q, m) ∧ p≠NULL
    node* x = p->next;
    // p↦x * list(x, n−1) * list(q, m) ∧ p≠NULL
    node* y = concat(x, q);
    // p↦x * list(y, n+m−1) ∧ p≠NULL
    p->next = y;
    // p↦y * list(y, n+m−1) ∧ p≠NULL
    return p;
    // p↦y * list(y, n+m−1) ∧ p≠NULL ∧ res=p
  }}
```

$\text{list}(u, n) \triangleq \text{emp} \wedge u{=}\texttt{NULL} \wedge n{=}0 \vee \exists v.(u{\mapsto}v * \text{list}(v, n{-}1))$

[i] Prove post-condition

$\text{list}(p, n) * \text{list}(q, m) \wedge p{=}\texttt{NULL} \wedge res{=}q \vdash \text{list}(res, n{+}m)$

[ii] Prove pre-condition

$\text{list}(x, n{-}1) * \text{list}(q, m) \vdash \exists n', m'.(\text{list}(x, n') * \text{list}(q, m'))$

[iii] Utilize post-condition

$\text{list}(res', n{+}m{-}1)$

[iv] Prove post-condition

$p{\mapsto}y * \text{list}(y, n{+}m{-}1) \wedge p{\neq}\texttt{NULL} \wedge res{=}p \vdash \text{list}(res, n + m)$

Separation Logic Entailments as Verification Conditions

- Verification conditions introduced when symbolically analyzing a program using separation logic are also called *entailments*.

Separation Logic Entailments as Verification Conditions

- Verification conditions introduced when symbolically analyzing a program using separation logic are also called *entailments*.
- Typical scenarios where an entailment is introduced:
  - Proving a *pre-condition* of a procedure.
  - Proving a *post-condition* of a procedure.
  - Normalizing, simplifying a program state.

Separation Logic Entailments as Verification Conditions

- Verification conditions introduced when symbolically analyzing a program using separation logic are also called *entailments*.
- Typical scenarios where an entailment is introduced:
  - Proving a *pre-condition* of a procedure.
  - Proving a *post-condition* of a procedure.
  - Normalizing, simplifying a program state.
- Examples:
  - $\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$
  - $\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$
  - $p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

The Focuses of this Thesis

Thesis statement

Advancing the entailment proving technique in the fragment of symbolic-heap separation logic with inductive predicates and linear arithmetic constraints.

The Focuses of this Thesis

Thesis statement

Advancing the entailment proving technique in the fragment of symbolic-heap separation logic with inductive predicates and linear arithmetic constraints.

- The symbolic-heap separation logic with inductive predicates is the most popular separation logic fragment. It is utilized by industrial verification tools like Facebook's Infer and Microsoft's SLAYer.

The Focuses of this Thesis

Thesis statement

Advancing the entailment proving technique in the fragment of symbolic-heap separation logic with inductive predicates and linear arithmetic constraints.

- ▷ The symbolic-heap separation logic with inductive predicates is the most popular separation logic fragment. It is utilized by industrial verification tools like Facebook's Infer and Microsoft's SLAYer.
- ▷ Existing researches mainly focus on the equality, but not the arithmetic constraints.

Outline

(1) Background

(2) Proving Entailments by Mutual Induction

(3) Proving Entailments by Synthesizing Lemmas

(4) Proving Entailments by Learning from Failure

(5) Discussion, Future Works, & Conclusion

Background

Syntax of Symbolic-Heap Separation Logic Formulas

c, x, ι, P: constants, variables, data sorts, and predicate symbols.

e	$::=$	$c \mid x \mid -e \mid e_1{+}e_2 \mid e_1{-}e_2 \mid c \cdot e$	Integer expression
a	$::=$	$\mathsf{nil} \mid x$	Spatial expression
t	$::=$	$e \mid a$	Term
π	$::=$	$\mathsf{true} \mid \mathsf{false} \mid a_1{=}a_2 \mid a_1{\neq}a_2 \mid e_1{=}e_2 \mid e_1{\neq}e_2$	Atomic pure formula
		$\mid e_1{>}e_2 \mid e_1{\geq}e_2 \mid e_1{<}e_2 \mid e_1{\leq}e_2$	
Π	$::=$	$\pi \mid \neg\Pi \mid \Pi_1{\wedge}\Pi_2 \mid \Pi_1{\vee}\Pi_2 \mid \Pi_1{\rightarrow}\Pi_2 \mid \forall x.\Pi \mid \exists x.\Pi$	Pure formula
σ	$::=$	$\mathsf{emp} \mid x{\overset{\iota}{\mapsto}}t_1,\ldots,t_n \mid \mathsf{P}(t_1,\ldots,t_n)$	Atomic spatial formula
Σ	$::=$	$\sigma \mid \Sigma_1 * \Sigma_2$	Spatial formula
F	$::=$	$\Sigma \mid \Pi \mid \Sigma \wedge \Pi \mid \exists x.F$	Symbolic-heap formula

Semantics of Symbolic-Heap Separation Logic Formulas

- A model of a separation logic formula is a pair s, h, where:
  - s is a stack model, describing values of variables
  - h is a heap model, presenting content of the memory.
- Satisfiability relation of separation logic model:

$s, h \models \Pi$	iff	$[\![\Pi]\!]_s = \text{true}$; and $\mathrm{dom}(h) = \varnothing$
$s, h \models \mathsf{emp}$	iff	$\mathrm{dom}(h) = \varnothing$
$s, h \models x \overset{\iota}{\mapsto} t_1, ..., t_n$	iff	$\mathrm{dom}(h) = \{s(x)\}$; and $h(s(x))(\iota) = (s(t_1), ..., s(t_n))$
$s, h \models \mathsf{P}(t_1, ..., t_n)$	iff	$s, h \models F_i(t_1, ..., t_n)$, where $F_i(t_1, ..., t_n) \overset{\text{def}}{\Rightarrow} \mathsf{P}(t_1, ..., t_n)$
$s, h \models \Sigma_1 * \Sigma_2$	iff	$\exists h_1, h_2.\ h_1 \# h_2; h_1 \circ h_2 = h; s, h_1 \models \Sigma_1$; and $s, h_2 \models \Sigma_2$
$s, h \models \Sigma \wedge \Pi$	iff	$[\![\Pi]\!]_s = \text{true}$; and $s, h \models \Sigma$
$s, h \models \exists x.\, F$	iff	$\exists v \in \mathtt{Val}.\ [s\vert x{:}v], h \models F$

Separation Logic Entailments

- *Definition:* an entailment between two formulas F_1 and F_2, denoted as $F_1 \vdash F_2$, is said to be *valid*, iff $s, h \models F_1$ implies that $s, h \models F_2$, for all models s, h.

Separation Logic Entailments

- *Definition:* an entailment between two formulas F_1 and F_2, denoted as $F_1 \vdash F_2$, is said to be *valid*, iff $s, h \models F_1$ implies that $s, h \models F_2$, for all models s, h.
- F_1 and F_2 are called the *antecedent* and the *consequent*.

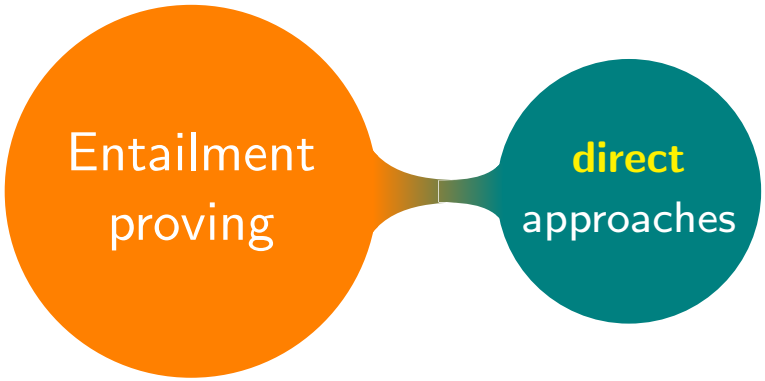
Separation Logic Entailments

- *Definition:* an entailment between two formulas F_1 and F_2, denoted as $F_1 \vdash F_2$, is said to be *valid*, iff $s, h \models F_1$ implies that $s, h \models F_2$, for all models s, h.
- F_1 and F_2 are called the *antecedent* and the *consequent*.
- Examples:
  - $\mathsf{list}(p, n) * \mathsf{list}(q, m) \wedge p{=}\mathtt{NULL} \wedge res{=}q \vdash \mathsf{list}(res, n{+}m)$
  - $\mathsf{list}(x, n{-}1) * \mathsf{list}(q, m) \vdash \exists n', m'.(\mathsf{list}(x, n') * \mathsf{list}(q, m'))$
  - $p{\mapsto}y * \mathsf{list}(y, n{+}m{-}1) \wedge p{\neq}\mathtt{NULL} \wedge res{=}p \vdash \mathsf{list}(res, n + m)$

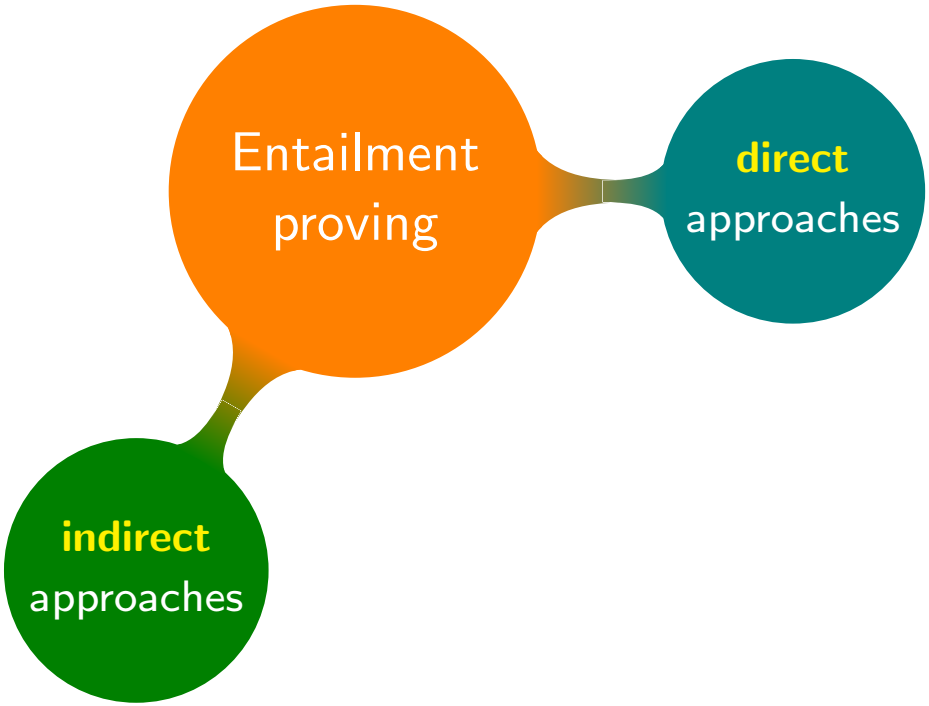
Existing Works on Proving Separation Logic Entailments

Existing Works on Proving Separation Logic Entailments

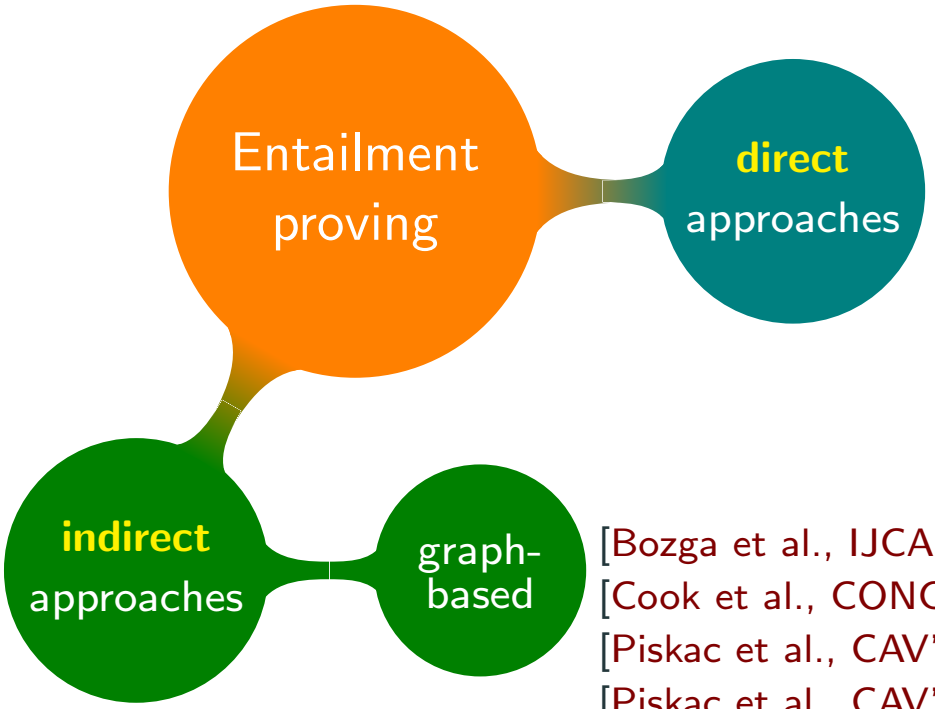
Existing Works on Proving Separation Logic Entailments



Existing Works on Proving Separation Logic Entailments

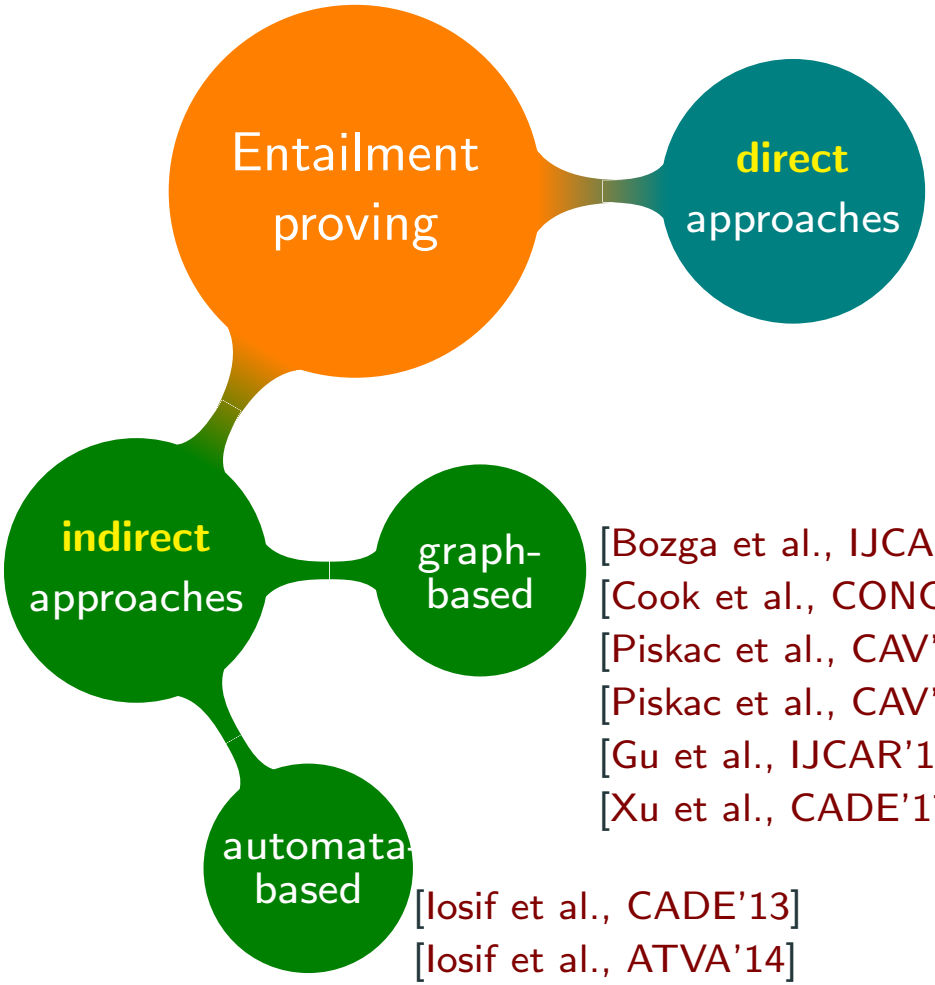


Existing Works on Proving Separation Logic Entailments



[Bozga et al., IJCAR'08]
[Cook et al., CONCUR'11]
[Piskac et al., CAV'13]
[Piskac et al., CAV'14]
[Gu et al., IJCAR'16]
[Xu et al., CADE'17]

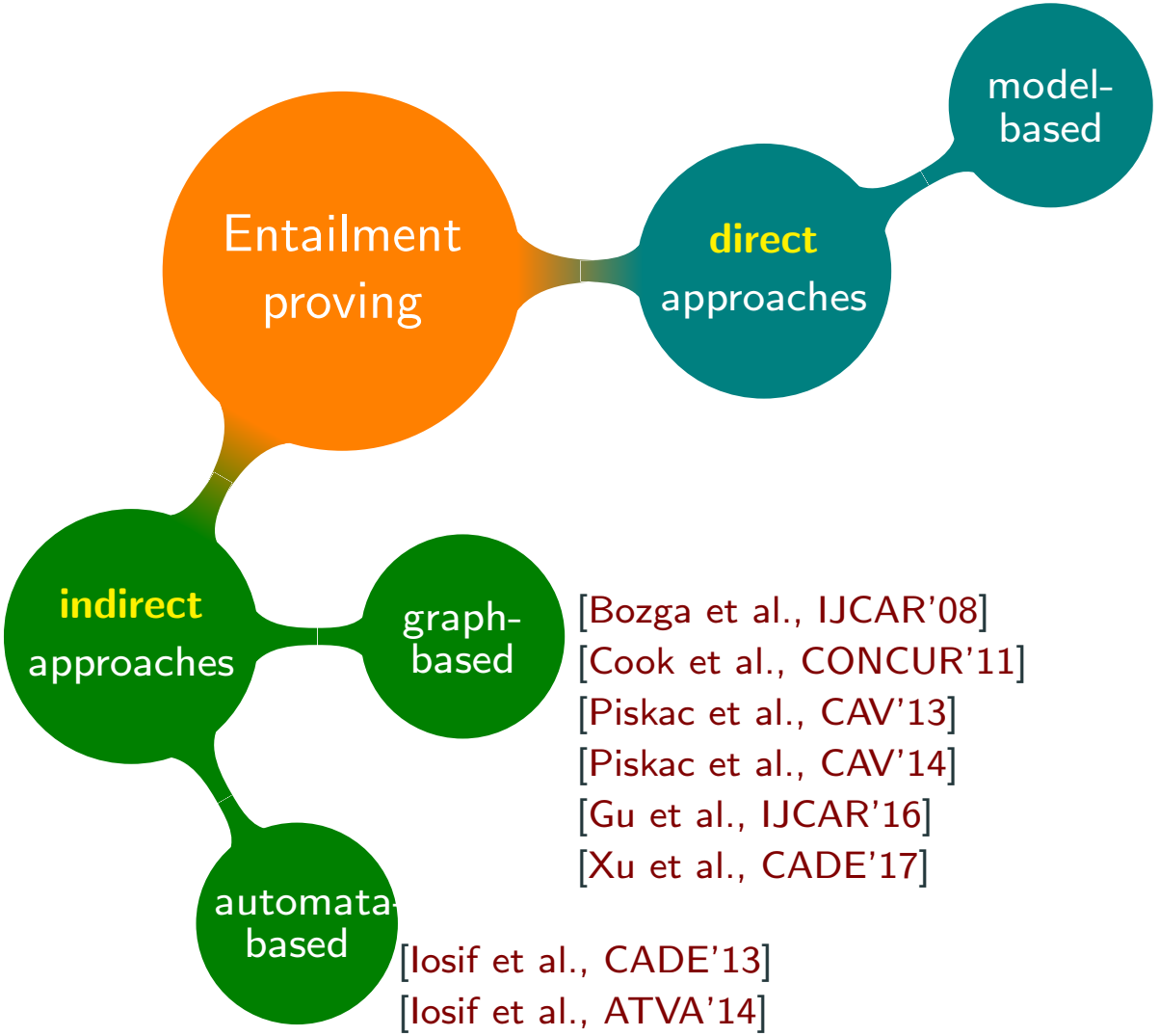
Existing Works on Proving Separation Logic Entailments



[Bozga et al., IJCAR'08]
[Cook et al., CONCUR'11]
[Piskac et al., CAV'13]
[Piskac et al., CAV'14]
[Gu et al., IJCAR'16]
[Xu et al., CADE'17]

[Iosif et al., CADE'13]
[Iosif et al., ATVA'14]
[Enea et al., APLAS'14]
[Enea et al., ATVA'15]

Existing Works on Proving Separation Logic Entailments

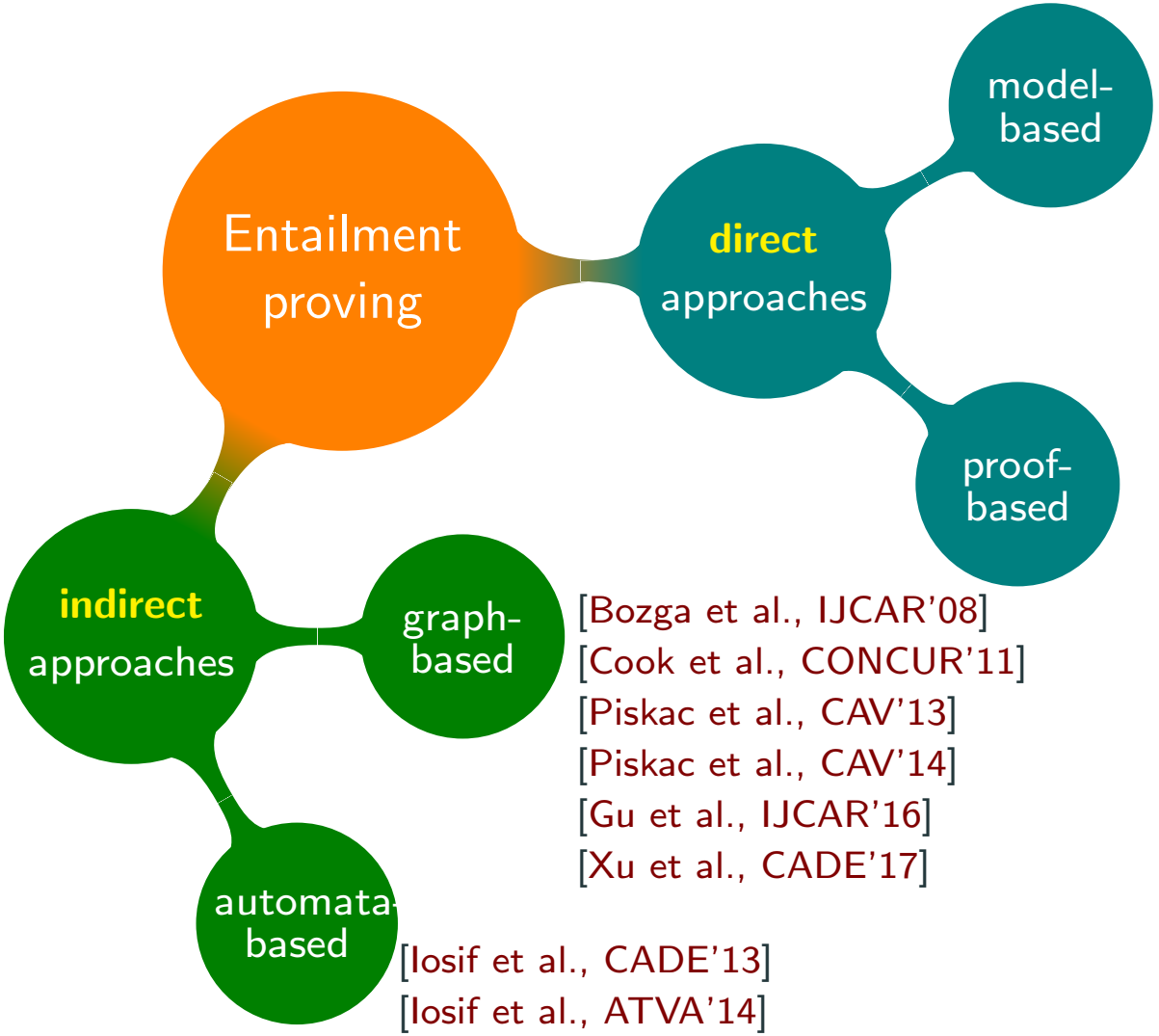


[Pérez et al., APLAS'13]

[Bozga et al., IJCAR'08]
[Cook et al., CONCUR'11]
[Piskac et al., CAV'13]
[Piskac et al., CAV'14]
[Gu et al., IJCAR'16]
[Xu et al., CADE'17]

[Iosif et al., CADE'13]
[Iosif et al., ATVA'14]
[Enea et al., APLAS'14]
[Enea et al., ATVA'15]

Existing Works on Proving Separation Logic Entailments

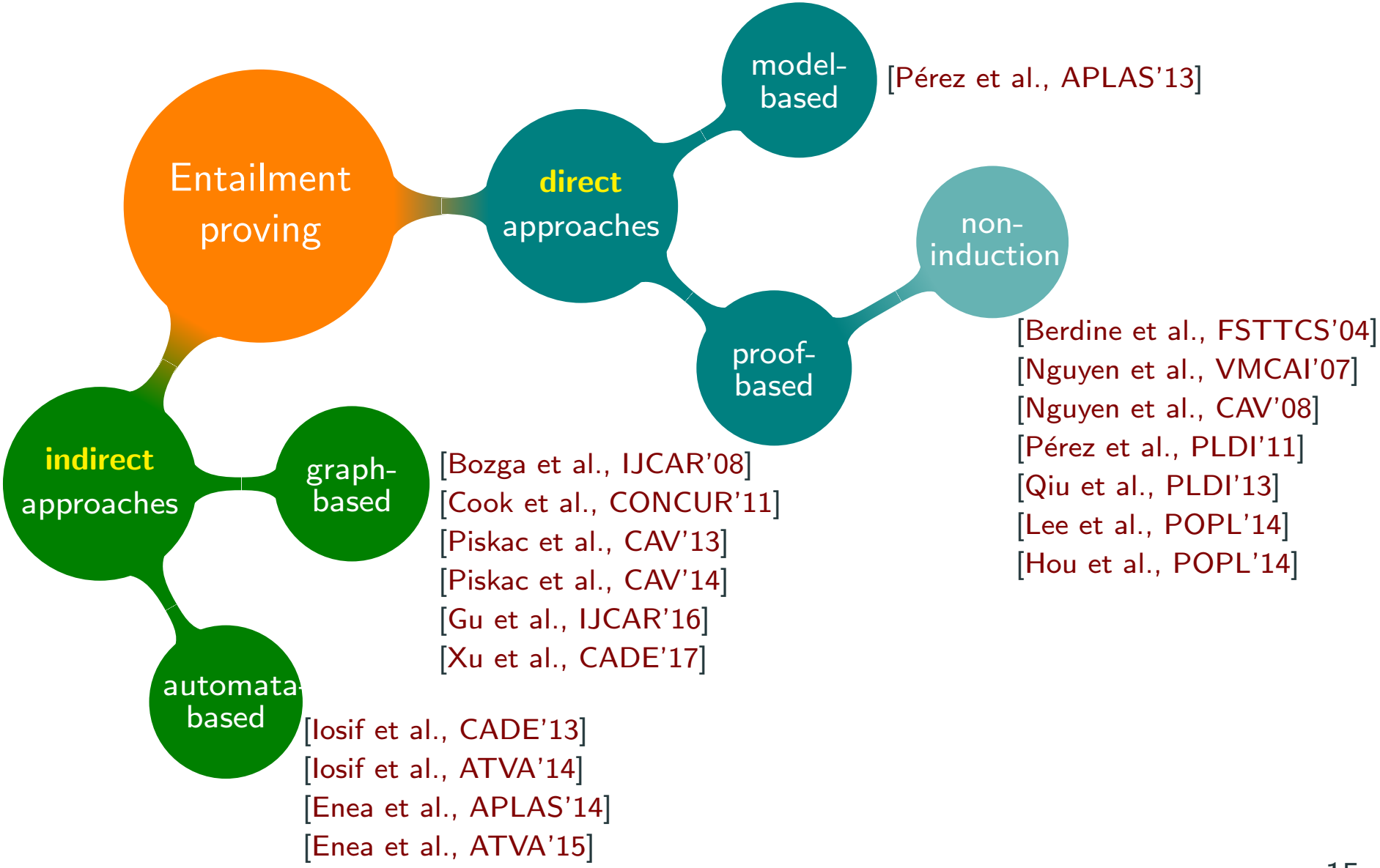


[Pérez et al., APLAS'13]

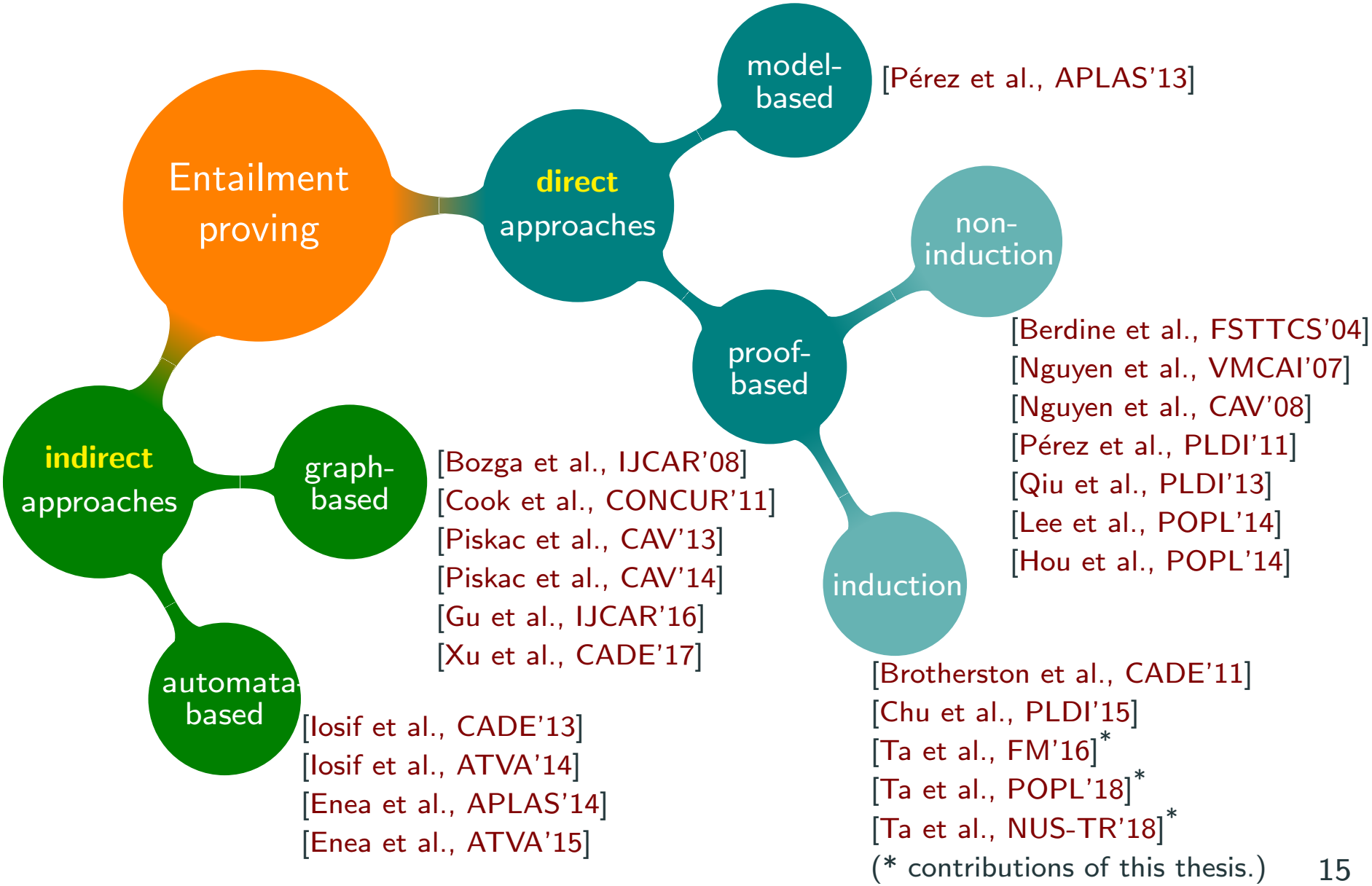
[Bozga et al., IJCAR'08]
[Cook et al., CONCUR'11]
[Piskac et al., CAV'13]
[Piskac et al., CAV'14]
[Gu et al., IJCAR'16]
[Xu et al., CADE'17]

[Iosif et al., CADE'13]
[Iosif et al., ATVA'14]
[Enea et al., APLAS'14]
[Enea et al., ATVA'15]

Existing Works on Proving Separation Logic Entailments



Existing Works on Proving Separation Logic Entailments



Proving Entailments by Mathematical Induction

Proving Entailments by Mathematical Induction

- High school: prove that a conjecture $\mathcal{P}$ is true for all natural numbers:

Proving Entailments by Mathematical Induction

- High school: prove that a conjecture $\mathcal{P}$ is true for all natural numbers:
  - Base case: verify $\mathcal{P}$ holds for 0, i.e., $\mathcal{P}(0)$.

Proving Entailments by Mathematical Induction

- High school: prove that a conjecture $\mathcal{P}$ is true for all natural numbers:
  - Base case: verify $\mathcal{P}$ holds for 0, i.e., $\mathcal{P}(0)$.
  - Inductive case: prove that for all n: $\mathcal{P}(n) \rightarrow \mathcal{P}(n{+}1)$.

Proving Entailments by Mathematical Induction

- High school: prove that a conjecture $\mathcal{P}$ is true for all natural numbers:
  - Base case: verify $\mathcal{P}$ holds for 0, i.e., $\mathcal{P}(0)$.
  - Inductive case: prove that for all n: $\mathcal{P}(n) \to \mathcal{P}(n{+}1)$.

  $\Rightarrow$ Peano induction principle for natural numbers:

$$\frac{\mathcal{P}(0) \qquad \forall n : nat.\ \mathcal{P}(n) \to \mathcal{P}(n+1)}{\forall n : nat.\ \mathcal{P}(n)}$$

Proving Entailments by Mathematical Induction

- High school: prove that a conjecture $\mathcal{P}$ is true for all natural numbers:
  - Base case: verify $\mathcal{P}$ holds for 0, i.e., $\mathcal{P}(0)$.
  - Inductive case: prove that for all n: $\mathcal{P}(n) \to \mathcal{P}(n{+}1)$.

  $\Rightarrow$ Peano induction principle for natural numbers:

$$\frac{\mathcal{P}(0) \qquad \forall n : nat.\ \mathcal{P}(n) \to \mathcal{P}(n+1)}{\forall n : nat.\ \mathcal{P}(n)}$$

- Noetherian induction for well-founded structures:

$$\frac{\forall \alpha : \tau.\ (\forall \beta : \tau \prec \alpha.\ \mathcal{P}(\beta) \to \mathcal{P}(\alpha))}{\forall \alpha : \tau.\ \mathcal{P}(\alpha)}$$

Proving Entailments by Mathematical Induction

- High school: prove that a conjecture $\mathcal{P}$ is true for all natural numbers:
  - Base case: verify $\mathcal{P}$ holds for 0, i.e., $\mathcal{P}(0)$.
  - Inductive case: prove that for all n: $\mathcal{P}(n) \rightarrow \mathcal{P}(n{+}1)$.

  $\Rightarrow$ Peano induction principle for natural numbers:

$$\frac{\mathcal{P}(0) \qquad \forall n : nat.\ \mathcal{P}(n) \rightarrow \mathcal{P}(n+1)}{\forall n : nat.\ \mathcal{P}(n)}$$

- Noetherian induction for well-founded structures:

$$\frac{\forall \alpha : \tau.\ (\forall \beta : \tau \prec \alpha.\ \mathcal{P}(\beta) \rightarrow \mathcal{P}(\alpha))}{\forall \alpha : \tau.\ \mathcal{P}(\alpha)}$$

- In this work: we utilize mathematical induction to reason about inductive heap predicates in entailments.

Proving Separation Logic Entailment by Mutual Induction

Motivating Example

- Consider a predicate ls modelling the linked list data structure:

$$\begin{aligned} \mathsf{ls}(x, y) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}y \\ & \vee\ \exists w.(x{\mapsto}w * \mathsf{ls}(w, y)) \end{aligned}$$

Motivating Example

- Consider a predicate ls modelling the linked list data structure:

$$\begin{aligned} \mathsf{ls}(x, y) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}y \\ & \vee\ \exists w.(x{\mapsto}w * \mathsf{ls}(w, y)) \end{aligned}$$

- And another predicate list modelling a variant of the linked list:

$$\begin{aligned} \mathsf{list}(x) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}\mathtt{NULL} \\ & \vee\ \exists u.(x{\mapsto}u * \mathsf{list}(u)) \\ & \vee\ \exists u, v.(x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v)) \end{aligned}$$

Motivating Example

- Consider a predicate ls modelling the linked list data structure:

$$\begin{aligned} \mathsf{ls}(x, y) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}y \\ & \vee\ \exists w.(x{\mapsto}w * \mathsf{ls}(w, y)) \end{aligned}$$

- And another predicate list modelling a variant of the linked list:

$$\begin{aligned} \mathsf{list}(x) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}\mathtt{NULL} \\ & \vee\ \exists u.(x{\mapsto}u * \mathsf{list}(u)) \\ & \vee\ \exists u, v.(x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v)) \end{aligned}$$

- Question: is the variant $\mathsf{list}(x)$ also a linked list segment $\mathsf{ls}(x, y)$?

Motivating Example

- Consider a predicate ls modelling the linked list data structure:

$$\begin{aligned} \mathsf{ls}(x, y) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}y \\ & \vee\ \exists w.(x{\mapsto}w * \mathsf{ls}(w, y)) \end{aligned}$$

- And another predicate list modelling a variant of the linked list:

$$\begin{aligned} \mathsf{list}(x) \quad \triangleq \quad & \mathsf{emp} \wedge x{=}\mathtt{NULL} \\ & \vee\ \exists u.(x{\mapsto}u * \mathsf{list}(u)) \\ & \vee\ \exists u, v.(x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v)) \end{aligned}$$

- Question: is the variant $\mathsf{list}(x)$ also a linked list segment $\mathsf{ls}(x, y)$?

  $\Rightarrow$ Prove the entailment: $\mathsf{list}(x) \vdash \exists y.\mathsf{ls}(x, y)$

Prove $\mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ by Structural Induction

- Record $E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ as an induction hypothesis.

Prove $\mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ by Structural Induction

- Record $E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ as an induction hypothesis.
- Unfold $\mathsf{list}(x)$ in the antecedent of E:

$$\dfrac{\dfrac{[\texttt{base step}]}{(E_1)}}{\mathsf{true} \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{\dfrac{[\texttt{inductive step}]}{(E_2)}}{x \mapsto u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{\dfrac{[\texttt{inductive step}]}{(E_3)}}{x \mapsto u * u \mapsto v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x, y)}$$
$$\overline{E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)}\ \mathsf{ID}$$

Prove $\mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ by Structural Induction

- Record $E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ as an induction hypothesis.
- Unfold $\mathsf{list}(x)$ in the antecedent of E:

$$\dfrac{\dfrac{[\texttt{base step}]}{(E_1)}}{\mathsf{true} \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{\dfrac{[\texttt{inductive step}]}{(E_2)}}{x \mapsto u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{\dfrac{[\texttt{inductive step}]}{(E_3)}}{x \mapsto u * u \mapsto v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x, y)}$$
$$\overline{E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)}\ \mathsf{ID}$$

Prove $\mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ by Structural Induction

- Record $E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$ as an induction hypothesis.
- Unfold $\mathsf{list}(x)$ in the antecedent of E:

$$\dfrac{\dfrac{[\texttt{base step}]}{(E_1)}}{\mathsf{true} \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{\dfrac{[\texttt{inductive step}]}{(E_2)}}{x \mapsto u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{\dfrac{[\texttt{inductive step}]}{(E_3)}}{x \mapsto u * u \mapsto v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x, y)}$$
$$\overline{E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)}\ \mathsf{ID}$$

Inductive Step (E_2) : $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Proof tree of E_2, by using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_{\pi}\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.(x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply IH } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{ID}
$$

Inductive Step (E_2) : $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Proof tree of E_2, by using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.(x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply IH } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{ID}
$$

Inductive Step (E_2) : $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Proof tree of E_2, by using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.(x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply IH } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID}
$$

Inductive Step (E_2) : $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Proof tree of E_2, by using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad \dfrac{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply IH } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID}
$$

$$
\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_{\pi}\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.(x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w
$$

$$
\mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)
$$

Inductive Step (E_2) : $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Proof tree of E_2, by using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_{\pi}\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.(x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply IH } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{ID}
$$

Inductive Step (E_2) : $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Proof tree of E_2, by using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.(x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply IH } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{ID}
$$

Inductive Step (E_3) : $x \mapsto u * u \mapsto v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x, y)$

$$\dfrac{\dfrac{(\textit{base step})\ (E_1)}{\mathsf{true} \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{(\textit{inductive step})\ (E_2)}{x \mapsto u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x, y)} \quad \dfrac{(\textit{inductive step})\ (E_3)}{x \mapsto u * u \mapsto v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x, y)}}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x, y)}\ \mathsf{ID}$$

Inductive Step (E_3) : $x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x, y)$

▷ Similar proof tree of E_3, using the induction hypothesis E:

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, z, w.\, (u{=}z \wedge v{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(v,t) \vdash \exists y, z, w.\, (\mathsf{ls}(w,y) \wedge u{=}z \wedge v{=}w)}\ *\text{P: Match } \mathsf{ls}(v,t) \text{ \& } \mathsf{ls}(w,y)}{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z, w.\, (z{\mapsto}w * \mathsf{ls}(w,y) \wedge u{=}z)}\ *{\mapsto}\text{: Match } u{\mapsto}v \text{ \& } z{\mapsto}w}{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z.\, (\mathsf{ls}(z,y) \wedge u{=}z)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(z,y)}{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z.\, (x{\mapsto}z * \mathsf{ls}(z,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}z}{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

Better Strategy: Prove E_3 by Reusing the Proof of E_2

- Once the proof of E_2 is found, we can collect entailments derived in this proof:

$(E_{2'})$ $x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)$

(E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)$

$$\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}$$

Better Strategy: Prove E_3 by Reusing the Proof of E_2

- Once the proof of E_2 is found, we can collect entailments derived in this proof:

$(E_{2'})$ $x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)$

(E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)$

$$\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}$$

- And use them as hypotheses to shorten the proof of E_3:

$$\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y)}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}$$

Better Strategy: Prove E_3 by Reusing the Proof of E_2

- Once the proof of E_2 is found, we can collect entailments derived in this proof:

$(E_{2'})$ $x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)$

(E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)$

$$\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}$$

- And use them as hypotheses to shorten the proof of E_3:

$$\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y)}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}$$

Better Strategy: Prove E_3 by Reusing the Proof of E_2

- Once the proof of E_2 is found, we can collect entailments derived in this proof:

$(E_{2'})$ $x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)$

(E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)$

$$\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}$$

- And use them as hypotheses to shorten the proof of E_3:

$$\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y)}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}$$

Better Strategy: Prove E_3 by Reusing the Proof of E_2

- Once the proof of E_2 is found, we can collect entailments derived in this proof:

$(E_{2'})$ $x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)$

(E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)$

$$\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E \text{ with } [u/x] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}$$

- And use them as hypotheses to shorten the proof of E_3:

$$\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y)}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}$$

Better Strategy: Prove E_3 by Reusing the Proof of E_2

- Once the proof of E_2 is found, we can collect entailments derived in this proof:

$$
\begin{array}{l}
(E_{2'})\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y) \\
(E_2)\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)
\end{array}
\qquad
\begin{array}{c}
\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)} \\
\dfrac{}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y) \\
\dfrac{}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w \\
\dfrac{}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y) \\
\dfrac{}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } E \text{ with } [u/x]
\end{array}
$$

$$
\dfrac{(E_1) \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}
$$

- And use them as hypotheses to shorten the proof of E_3:

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y)}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

Comparison: the New Proof and the Standard Proof of E_3

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \text{IH: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\,\mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y).}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y,z,w.\,(u{=}z \wedge v{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(v,t) \vdash \exists y,z,w.\,(\mathsf{ls}(w,y) \wedge u{=}z \wedge v{=}w)}\ *\text{P: Match } \mathsf{ls}(v,t) \text{ \& } \mathsf{ls}(w,y)}{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y,z,w.\,(z{\mapsto}w * \mathsf{ls}(w,y) \wedge u{=}z)}\ *{\mapsto}\text{: Match } u{\mapsto}v \text{ \& } z{\mapsto}w}{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y,z.\,(\mathsf{ls}(z,y) \wedge u{=}z)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(z,y)}{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y,z.\,(x{\mapsto}z * \mathsf{ls}(z,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}z}{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply IH } E}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

What is the Difference between the Two Proofs?

- The use of cut rule in the entailment proof:

$$\frac{\langle A \vdash B \rangle_{\text{cut-point}} \qquad F \vdash A * K \qquad B * K \vdash G}{F \vdash G}$$

What is the Difference between the Two Proofs?

- The use of cut rule in the entailment proof:

$$\frac{\langle A \vdash B \rangle_{\text{cut-point}} \qquad F \vdash A * K \qquad B * K \vdash G}{F \vdash G}$$

- Source of cut points:

What is the Difference between the Two Proofs?

- The use of cut rule in the entailment proof:

$$\frac{\langle A \vdash B \rangle_{\text{cut-point}} \qquad F \vdash A * K \qquad B * K \vdash G}{F \vdash G}$$

- Source of cut points:
  - Existing induction-based works: use only the set of induction hypotheses [Chu et al., PLDI'15].

What is the Difference between the Two Proofs?

- The use of cut rule in the entailment proof:

$$\frac{\langle A \vdash B \rangle_{\text{cut-point}} \qquad F \vdash A * K \qquad B * K \vdash G}{F \vdash G}$$

- Source of cut points:
  - Existing induction-based works: use only the set of induction hypotheses [Chu et al., PLDI'15].
  - This work: induction hypotheses + other derived entailments.

How about the Soundness?

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_{\pi}\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } H \text{ with } [u/a] \qquad (E_3)}{E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

▷ When finish proving E_2, we don't know about E_3 and E!

How about the Soundness?

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } H \text{ with } [u/a] \qquad (E_3)}{E \triangleq \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

- ▷ When finish proving E_2, we don't know about E_3 and E!
  - (E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y) \rightsquigarrow$ validity: **?**

How about the Soundness?

$$
\dfrac{(E_1) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)}\ \vdash_\pi\text{: SMT solver (Z3)}}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}\ *\text{P: Match } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}\ *{\mapsto}\text{: Match } x{\mapsto}u \text{ and } x{\mapsto}w}{E_{2'} \triangleq\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \mathsf{P_R}\text{: Unfold } \mathsf{ls}(x,y)}{E_2 \triangleq\ x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{IH: Apply } H \text{ with } [u/a] \qquad (E_3)}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\, \mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

- ▷ When finish proving E_2, we don't know about E_3 and E!
  - (E_2) $x{\mapsto}u * \mathsf{list}(u) \vdash \exists y.\, \mathsf{ls}(x,y) \rightsquigarrow$ validity: **?**
  - $(E_{2'})$ $x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y) \rightsquigarrow$ validity: ✓

How about the Soundness?

▷ Using $E_2(?)$ as a hypothesis to prove E_3:

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\,\mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y).}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

How about the Soundness?

▷ Using $E_2(?)$ as a hypothesis to prove E_3:

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\,\mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y).}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

⇒ How to conclude about E_3 and then E?

How about the Soundness?

▷ Using $E_2(?)$ as a hypothesis to prove E_3:

$$
\dfrac{(E_1) \qquad (E_2) \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists y.(y{=}z)}\ \vdash_\pi\text{: SMT solver}}{\mathsf{ls}(x,z) \vdash \exists y.\,\mathsf{ls}(x,y)}\ *\text{P: Match } \mathsf{ls}(x,z), \mathsf{ls}(x,y).}{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_{2'}}{E_3 \triangleq\ x{\mapsto}u * u{\mapsto}v * \mathsf{list}(v) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{IH: Apply HP } E_2}{E \triangleq\ \mathsf{list}(x) \vdash \exists y.\,\mathsf{ls}(x,y)}\ \text{ID: Induction}
$$

⇒ How to conclude about E_3 and then E?

⇒ Consider the mutual inductive proof of E, E_2, E_3!

The Proof System: General Inference Rules

$$\bot_{\mathsf{L}}^{\pi}\ \frac{}{\mathcal{H},\ \rho,\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \Pi_1 \rightarrow \mathsf{false}$$

$$\bot_{\mathsf{L}}^{\sigma}\ \frac{}{\mathcal{H},\ \rho,\ \Sigma_1 * u \overset{\iota_1}{\mapsto} \vec{t} * u \overset{\iota_2}{\mapsto} \vec{r} \wedge \Pi_1 \vdash F_2}$$

$$\vdash_{\pi}\ \frac{}{\mathcal{H},\ \rho,\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2$$

$$=_{\mathsf{L}}\ \frac{\mathcal{H},\ \rho',\ (\Sigma \wedge \Pi_1)[t/x] \vdash F_2[t/x]}{\mathcal{H},\ \rho,\ \Sigma_1 \wedge \Pi_1 \wedge t{=}x \vdash F_2}$$

$$*\mapsto\ \frac{\mathcal{H},\ \rho',\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2 \wedge u{=}w \wedge \vec{t}{=}\vec{r})}{\mathcal{H},\ \rho,\ \Sigma_1 * u \overset{\iota}{\mapsto} \vec{t} \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 * w \overset{\iota}{\mapsto} \vec{r} \wedge \Pi_2)}\ \vec{x} \# \mathtt{FV}(u, \vec{t})$$

$$\exists_{\mathsf{L}}\ \frac{\mathcal{H},\ \rho',\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \rho,\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2)$$

$$*\mathsf{P}\ \frac{\mathcal{H},\ \rho',\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2 \wedge \vec{t}{=}\vec{r})}{\mathcal{H},\ \rho,\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{P}(\vec{r}) \wedge \Pi_2)}\ \vec{x} \# \mathtt{FV}(\vec{t})$$

$$\exists_{\mathsf{R}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.((\Sigma_2 \wedge \Pi_2)[t/y])}{\mathcal{H},\ \rho,\ F_1 \vdash \exists \vec{x}, y.(\Sigma_2 \wedge \Pi_2 \wedge t{=}y)}$$

$$\mathsf{P}_{\mathsf{L}}\ \frac{\mathcal{H},\ \rho',\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \rho,\ \Sigma_1 * \Sigma' \wedge \Pi_1 \wedge \Pi' \vdash F_2}\ \exists \vec{x}.(\Sigma' \wedge \Pi') \overset{\text{def}}{\Rightarrow} \mathsf{P}(\vec{t})$$

$$\mathsf{E}_{\mathsf{L}}\ \frac{\mathcal{H},\ \rho',\ \Sigma_1 \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \rho,\ \Sigma_1 * \mathsf{emp} \wedge \Pi_1 \vdash F_2}$$

$$\mathsf{P}_{\mathsf{R}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}, \vec{y}.(\Sigma_2 * \Sigma' \wedge \Pi_2 \wedge \Pi')}{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{P}(\vec{t}) \wedge \Pi_2)}\ \exists \vec{y}.(\Sigma' \wedge \Pi') \overset{\text{def}}{\Rightarrow} \mathsf{P}(\vec{t})$$

$$\mathsf{E}_{\mathsf{R}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2)}{\mathcal{H},\ \rho,\ F_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{emp} \wedge \Pi_2)}$$

($\mathcal{H}$ is a set of induction hypotheses, ρ is a proof trace of the corresponding entailment)

The Proof System: Induction Rules

$$\mathsf{ID}\ \frac{\mathcal{H}',\ \rho',\ \Sigma_1 * \Sigma_1' \wedge \Pi_1 \wedge \Pi_1' \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ \rho',\ \Sigma_1 * \Sigma_n' \wedge \Pi_1 \wedge \Pi_n' \vdash F_2}{\mathcal{H},\ \rho,\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger$: $\mathcal{H}' \triangleq \mathcal{H} \cup \{(H, \mathbf{?})\}$, where H is obtained by renaming vars of $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$, $\mathsf{P}(\vec{t}) \stackrel{\text{def}}{=} \exists \vec{x}_1.(\Sigma_1' \wedge \Pi_1') \vee \ldots \vee \exists \vec{x}_n.(\Sigma_n' \wedge \Pi_n')$, and $\mathtt{FV}(\vec{x}_1, ..., \vec{x}_n) \# \mathtt{FV}(\Sigma_1, \Pi_1)$.

$$\mathsf{IH}\ \frac{\mathcal{H} \cup \{(H, \kappa)\},\ \rho',\ \Sigma * \Sigma_4\theta \wedge \Pi_1 \wedge \Pi_4\theta \vdash F_2}{\mathcal{H} \cup \{(H \triangleq \Sigma_3 \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4), \kappa)\},\ \rho,\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger$: $\exists \theta, \Sigma.((\Sigma_1 \cong \Sigma_3\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta))$, $\vec{x} \# \mathtt{FV}(\Sigma_1, \Pi_1)$, and

$$(\kappa = \checkmark) \vee \exists\, \iota, u, \vec{r}, \Sigma.(\Sigma \cong u \stackrel{\iota}{\mapsto} \vec{r} * \Sigma)$$
$$\vee \exists\, \rho_1, \rho_2.(\rho = \rho_1 @ [(* \mapsto)] @ \rho_2 \wedge \mathsf{ID} \notin \rho_1 \wedge \mathsf{ID} \in \rho_2)$$

Implementation and Experiment

- Implementation:
  - The separation logic inductive prover: Songbird.
  - https://songbird-prover.github.io/mutual-induction
- Experiment:
  - Benchmarks: slrd_entl (SL-COMP 2014), slrd_indt (synthetic).
  - Environment: Ubuntu 14.04, Intel E5-2620 (2.4GHz), RAM 64GB.
  - Date conducted: May 2016.

Experiment: Benchmark slrd_entl (SL-COMP 2014)

Category	Slide	Spen	Sleek	Cyclist	Songbird
singly-ll (64)	12	3	48	**63**	**63**
doubly-ll (37)	14	0	17	24	**26**
nested-ll (11)	0	**11**	5	6	**11**
skip-list (13)	0	**12**	4	5	7
tree (26)	12	1	14	18	**22**
Total (151)	38	27	88	116	**129**

Overall comparison on number of proved entailments (Timeout: 30s)

Slide: [Iosif et al., CADE'13]
Spen: [Enea et al., APLAS'14]
Sleek: [Chin et al., SCP'12]
Cyclist: [Brotherston et al., CADE'11]
Songbird: [this work]

	Songbird			
	$\checkmark_{sb}\,\times_{o}$	$\times_{sb}\,\checkmark_{o}$	$\checkmark_{sb}\,\checkmark_{o}$	$\times_{sb}\,\times_{o}$
Cyclist	13	0	116	22
Sleek	41	0	88	22
Spen	109	7	20	15
Slide	103	12	26	10

Detailed comparison

Experiment: Benchmark slrd_indt (Synthetic)

Category		Cyclist	SongbirdSI	Songbird
ll/ll2	(24)	18	22	**24**
ll-even/odd	(20)	8	17	**20**
ll-left/right	(20)	12	10	**20**
misc.	(32)	17	16	**32**
Total	(96)	55	65	**96**

Number of proved entailments (Timeout: 30s)

Note: **SongbirdSI** is a variant of Songbird, whose mutual induction feature is disable, i.e., supporting only structural induction.

Proving Separation Logic Entailment by Synthesizing Lemmas

Overview of Existing Approaches

- ▷ Cyclic and induction proofs have been widely utilized to prove separation logic entailments: the mutual induction proof system + [Brotherston et al., CADE'11, Chu et al., PLDI'15].

Overview of Existing Approaches

- ▷ Cyclic and induction proofs have been widely utilized to prove separation logic entailments: the mutual induction proof system + [Brotherston et al., CADE'11, Chu et al., PLDI'15].
- ▷ However, existing entailment benchmarks [SL-COMP-14], [Ta et al., FM'16] are relatively simple and contain only equality constraints.

Overview of Existing Approaches

- Cyclic and induction proofs have been widely utilized to prove separation logic entailments: the mutual induction proof system + [Brotherston et al., CADE'11, Chu et al., PLDI'15].
- However, existing entailment benchmarks [SL-COMP-14], [Ta et al., FM'16] are relatively simple and contain only equality constraints.
- Can these techniques handle more sophisticated entailments, especially with arithmetic constraints?

Motivating Example

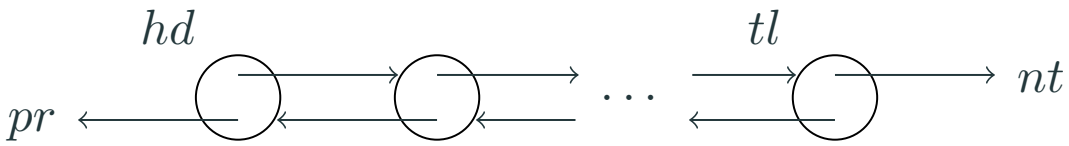
- Consider proving the following entailment:

$$E \triangleq \mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto t, r)$$

where

$$\mathsf{dll}(hd, pr, tl, nt, len) \stackrel{\text{def}}{=} hd \mapsto nt, pr \wedge hd = tl \wedge len = 1 \vee \exists u.(hd \mapsto u, pr * \mathsf{dll}(u, hd, tl, nt, len{-}1))$$

$$\mathsf{dllr}(hd, pr, tl, nt, len) \stackrel{\text{def}}{=} hd \mapsto nt, pr \wedge hd = tl \wedge len = 1 \vee \exists u.(\mathsf{dllr}(hd, pr, u, tl, len{-}1) * tl \mapsto nt, u)$$



Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

- Record it as an induction hypothesis:

  H : $\mathsf{dllr}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

- Record it as an induction hypothesis:

  H : $\mathsf{dllr}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$

- Unfold $\mathsf{dllr}(x, y, u, v, n)$ of E by inductive case:

  E' : $\mathsf{dllr}(x, y, w, u, n{-}1) * u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

- Record it as an induction hypothesis:

  H : $\mathsf{dllr}(a, b, p, \underline{q}, m) * \mathsf{dll}(\underline{q}, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$

- Unfold $\mathsf{dllr}(x, y, u, v, n)$ of E by inductive case:

  E' : $\mathsf{dllr}(x, y, w, \underline{u}, n{-}1) * u \mapsto v, w * \mathsf{dll}(\underline{v}, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply H to prove E': $\underline{q}$ is mapped to $\underline{u}$, $\underline{v}$ when matching dllr, dll.

Attempt to Prove E by Structural Induction...

- Original entailment:
  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:
  H : $\mathsf{dllr}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllr}(x, y, u, v, n)$ of E by inductive case:
  E' : $\mathsf{dllr}(x, y, w, u, n{-}1) * u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
  $\Rightarrow$ Cannot apply H to prove E': q is mapped to u, v when matching dllr, dll.
- Fold $u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200)$ of E' into $\mathsf{dll}(u, w, z, t, 201)$:
  E'' : $\mathsf{dllr}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

- Record it as an induction hypothesis:

  H : $\mathsf{dllr}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, \underline{200}) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$

- Unfold $\mathsf{dllr}(x, y, u, v, n)$ of E by inductive case:

  E' : $\mathsf{dllr}(x, y, w, u, n{-}1) * u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply H to prove E': q is mapped to u, v when matching dllr, dll.

- Fold $u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200)$ of E' into $\mathsf{dll}(u, w, z, t, 201)$:

  E'' : $\mathsf{dllr}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, \underline{201}) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply H to prove E'': $\underline{200}$ is mapped to $\underline{201}$ when matching dll.

Attempt to Prove E by Structural Induction...

- Original entailment:
  E : $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:
  H : $\mathsf{dllr}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllr}(x, y, u, v, n)$ of E by inductive case:
  E' : $\mathsf{dllr}(x, y, w, u, n{-}1) * u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
  $\Rightarrow$ Cannot apply H to prove E': q is mapped to u, v when matching dllr, dll.
- Fold $u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200)$ of E' into $\mathsf{dll}(u, w, z, t, 201)$:
  E'' : $\mathsf{dllr}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
  $\Rightarrow$ Cannot apply H to prove E'': 200 is mapped to 201 when matching dll.
- Structural induction fails to prove the entailment E. (So are the existing cyclic and induction proofs: the mutual induction proof system + [Brotherston et al., CADE'11, Chu et al., PLDI'15].)

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

  $\mathsf{dllr}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2$$

$$\dfrac{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\pi}\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas

- Original entailment:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_\pi\text{: SMT solver}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\text{P: match predicates}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_2}{\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \text{Apply } L_1$$

$$L_1 \triangleq \mathsf{dllr}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

- How to synthesize the auxiliary lemmas?

Our Approach to Synthesize Lemmas

- Given a goal entailment...

Our Approach to Synthesize Lemmas

- ▷ Given a goal entailment...
- ▷ Step 1: Discover lemma templates having similar heap structure with the entailment.

Our Approach to Synthesize Lemmas

- ▷ Given a goal entailment...
- ▷ Step 1: Discover lemma templates having similar heap structure with the entailment.
- ▷ Step 2: Refine the lemma templates by finding pure constraints over their variables to make the lemmas valid:

Our Approach to Synthesize Lemmas

- ▷ Given a goal entailment...
- ▷ Step 1: Discover lemma templates having similar heap structure with the entailment.
- ▷ Step 2: Refine the lemma templates by finding pure constraints over their variables to make the lemmas valid:
  - Synthesize constraints in the *antecedents*.

Our Approach to Synthesize Lemmas

- ▷ Given a goal entailment...
- ▷ Step 1: Discover lemma templates having similar heap structure with the entailment.
- ▷ Step 2: Refine the lemma templates by finding pure constraints over their variables to make the lemmas valid:
  - Synthesize constraints in the *antecedents*.
  - Synthesize constraints in the *consequents*.

Step 1: Discover the Lemma Templates

- ▷ Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).

Step 1: Discover the Lemma Templates

- ▷ Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).
- ▷ Example: the motivating entailment E:

$$\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$$

Step 1: Discover the Lemma Templates

- ▷ Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).
- ▷ Example: the motivating entailment E:

  $\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- ▷ Possible lemma templates:

Step 1: Discover the Lemma Templates

- ▷ Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).
- ▷ Example: the motivating entailment E:

  $\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- ▷ Possible lemma templates:

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$

Step 1: Discover the Lemma Templates

- Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).
- Example: the motivating entailment E:

  $\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- Possible lemma templates:

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$

  $\mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

Step 1: Discover the Lemma Templates

- Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).
- Example: the motivating entailment E:

  $\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- Possible lemma templates:

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$

  $\mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

Step 1: Discover the Lemma Templates

- Choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied).
- Example: the motivating entailment E:

  $\mathsf{dllr}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- Possible lemma templates:

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$

  $\mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

  $\mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) * \mathsf{dll}(x_5,x_6,x_7,x_8,n_2) \vdash \exists x_9,\ldots,n_3.\mathsf{dll}(x_9,x_{10},x_{11},x_{12},n_3)$

Step 2: Refine the Lemma Templates

- Discovered lemma templates from previous step:

  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

Step 2: Refine the Lemma Templates

- Discovered lemma templates from previous step:
  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
- This step: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).

Step 2: Refine the Lemma Templates

- Discovered lemma templates from previous step:
  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
- This step: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).
  - In practice, start from the simplest templates first.

Step 2: Refine the Lemma Templates

- Discovered lemma templates from previous step:

  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

- This step: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).
  - In practice, start from the simplest templates first.
  - Synthesize the templates' antecedents:

    $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, \ldots, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

Step 2: Refine the Lemma Templates

- Discovered lemma templates from previous step:

  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

- This step: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).
  - In practice, start from the simplest templates first.
  - Synthesize the templates' antecedents:

    $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, \ldots, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
  - Synthesize the templates' consequents:

    $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) \wedge ?)$

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,..,11}, n_2.(\mathsf{dll}(x_5, \ldots, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:

  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge \textbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:

  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge \textbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
- Refinement steps:

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:
  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge \textbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
- Refinement steps:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:
  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge \textbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
- Refinement steps:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).
  - Create an unknown entailment:
    E_{u_1} : $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge U(\vec{x}, \vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:
  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge \mathbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
- Refinement steps:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).
  - Create an unknown entailment:
    E_{u_1} : $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge U(\vec{x}, \vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  - *Prove* the unknown entailment E_{u_1} by induction and *collect* a constraint set of $U(\vec{x}, \vec{n})$.

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:
  $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge \mathbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
- Refinement steps:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).
  - Create an unknown entailment:
    E_{u_1} : $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge U(\vec{x}, \vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  - *Prove* the unknown entailment E_{u_1} by induction and *collect* a constraint set of $U(\vec{x}, \vec{n})$.
  - *Solve* the constraint set to find out the actual definition of $U(\vec{x}, \vec{n})$ and discover the lemma.

Step 2.1: Refine the Lemma Templates' Antecedents

▷ Prove the unknown entailment by induction:

$$
\dfrac{
\dfrac{
\dfrac{
\dfrac{
\dfrac{\mathcal{A}_1 \triangleq \{x_1{=}x_3 \land n_1{=}1 \land U(\vec{x},\vec{n}) \to x_1{=}x_5 \land x_2{=}x_6 \land x_4{=}x_8 \land x_5{=}x_7 \land n_2{=}1\}}{x_1{=}x_3 \land n_1{=}1 \land U(\vec{x},\vec{n}) \vdash x_1{=}x_5 \land x_2{=}x_6 \land x_4{=}x_8 \land x_5{=}x_7 \land n_2{=}1}\ \mathsf{U}_{\mathsf{L}}^{\pi}
}{x_1{\mapsto}x_2, x_4 \land x_1{=}x_3 \land n_1{=}1 \land U(\vec{x},\vec{n}) \vdash x_5{\mapsto}x_6, x_8 \land x_5{=}x_7 \land n_2{=}1}\ *{\mapsto}
}{x_1{\mapsto}x_2, x_4 \land x_1{=}x_3 \land n_1{=}1 \land U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\ \mathsf{P_R}
\quad
\dfrac{
\dfrac{
\dfrac{\mathcal{A}_2 \triangleq \{U(\vec{x},\vec{n}) \to \mathsf{false}\}}{x_3{\mapsto}u, x_4 \land U(\vec{x},\vec{n}) \vdash a_5{=}x_5 \land a_6{=}x_6 \land a_7{=}x_7 \land a_8{=}x_8 \land b_2{=}n_2}\ \mathsf{U}_{\mathsf{L}}^{\sigma}
}{\mathsf{dll}(a_5,a_6,a_7,a_8,b_2) * x_3{\mapsto}u, x_4 \land U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\ *\mathsf{P}
\quad
\mathcal{A}_3 \triangleq \{U(\vec{x},\vec{n}) \to U(\vec{a},\vec{b})\theta\}
}{\mathsf{dllr}(x_1,x_2,u,x_3,n_1{-}1) * x_3{\mapsto}u, x_4 \land U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\ \mathsf{U_{IH}}
}{E_{u_1} \triangleq \mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \land U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\ \mathsf{ID}
$$

(with $H \triangleq \mathsf{dllr}(a_1,...,a_4,b_1) \land U(\vec{a},\vec{b}) \vdash \mathsf{dll}(a_5,...,a_8,b_2)$, $\theta = [x_1/a_1, x_2/a_2, u/a_3, x_3/a_4, n_1{-}1/b_1]$)

Step 2.1: Refine the Lemma Templates' Antecedents

- Prove the unknown entailment by induction:

$$
\dfrac{
\dfrac{
\dfrac{
\dfrac{\mathcal{A}_1 \triangleq \{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1\}}{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,\mathsf{U}_{\mathsf{L}}^{\pi}
}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_5{\mapsto}x_6, x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,{*}{\mapsto}
}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{P_R}
\quad
\dfrac{
\dfrac{
\dfrac{\mathcal{A}_2 \triangleq \{U(\vec{x},\vec{n}) \rightarrow \mathsf{false}\}}{x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash a_5{=}x_5 \wedge a_6{=}x_6 \wedge a_7{=}x_7 \wedge a_8{=}x_8 \wedge b_2{=}n_2}\,\mathsf{U}_{\mathsf{L}}^{\sigma}
}{\mathsf{dll}(a_5,a_6,a_7,a_8,b_2) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,{*}\mathsf{P}
\quad
\mathcal{A}_3 \triangleq \{U(\vec{x},\vec{n}) \rightarrow U(\vec{a},\vec{b})\theta\}
}{\mathsf{dllr}(x_1,x_2,u,x_3,n_1{-}1) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{U_{IH}}
}{E_{u_1} \triangleq \mathsf{dllr}(x_1,x_2,x_3,x_4,n_1) \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{ID}
$$

(with $H \triangleq \mathsf{dllr}(a_1,...,a_4,b_1) \wedge U(\vec{a},\vec{b}) \vdash \mathsf{dll}(a_5,...,a_8,b_2)$, $\theta = [x_1/a_1, x_2/a_2, u/a_3, x_3/a_4, n_1{-}1/b_1]$)

- Collect a constraint set $\mathcal{A} = \mathcal{A}_1 \cup \mathcal{A}_2 \cup \mathcal{A}_3$ over the unknown relation U:

1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1,...,x_8,n_1,n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
2. $U(x_1,...,x_8,n_1,n_2) \rightarrow \mathsf{false}$
3. $U(x_1,...,x_8,n_1,n_2) \rightarrow U(x_1,x_2,u,x_3,a_5,a_6,a_7,a_8,n_1{-}1,b_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:

$$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Substitute the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Substitute the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.
  - Use SMT solvers to find out the unknown coefficients c_j^i, d_j^i.

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Substitute the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.
  - Use SMT solvers to find out the unknown coefficients c_j^i, d_j^i.
- Practical approaches:

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Substitute the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.
  - Use SMT solvers to find out the unknown coefficients c_j^i, d_j^i.
- Practical approaches:
  - Solve the *entire* set $\mathcal{A}$ to discover a *full solution*.

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Substitute the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.
  - Use SMT solvers to find out the unknown coefficients c_j^i, d_j^i.
- Practical approaches:
  - Solve the *entire* set $\mathcal{A}$ to discover a *full solution*.
  - Solve a *subset* of $\mathcal{A}$ to discover a *partial solution*.

Step 2.1: Refine the Lemma Templates' Antecedents

- The constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- The constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- When solving the full set $\mathcal{A}$, we discover only a spurious solution:
  $U(x_1, ..., x_8, n_1, n_2) \equiv$ false (✗ not useful!!)

Step 2.1: Refine the Lemma Templates' Antecedents

- The constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- When solving the full set $\mathcal{A}$, we discover only a spurious solution:
  $U(x_1, ..., x_8, n_1, n_2) \equiv$ false (✗ not useful!!)
- When solving a subset of $\mathcal{A}$ containing the first constraint (1):
  $\overline{U}(x_1, ..., x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$

Step 2.1: Refine the Lemma Templates' Antecedents

- The constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- When solving the full set $\mathcal{A}$, we discover only a spurious solution:
  $U(x_1, ..., x_8, n_1, n_2) \equiv$ false (✗ not useful!!)
- When solving a subset of $\mathcal{A}$ containing the first constraint (1):
  $\overline{U}(x_1, ..., x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$
  - Discover (and verify) the following lemma:
    $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$
    $\vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- The constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, ..., x_8, n_1, n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, ..., x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, ..., x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$
- When solving the full set $\mathcal{A}$, we discover only a spurious solution:
  $U(x_1, ..., x_8, n_1, n_2) \equiv$ false (✗ not useful!!)
- When solving a subset of $\mathcal{A}$ containing the first constraint (1):
  $\overline{U}(x_1, ..., x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$
  - Discover (and verify) the following lemma:
    $\mathsf{dllr}(x_1, x_2, x_3, x_4, n_1) \wedge x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2 \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  - Normalize to obtain the motivating lemma L_1:
    $L_1 \triangleq \mathsf{dllr}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$ (✓ desired!!)

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.
  $\Rightarrow$ refine in 2 phases.

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.
  $\Rightarrow$ refine in 2 phases.
- **Phase 1: preprocess the lemma template:**

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.
  $\Rightarrow$ refine in 2 phases.
- **Phase 1: preprocess the lemma template:**
  - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.
  $\Rightarrow$ refine in 2 phases.
- **Phase 1: preprocess the lemma template:**
  - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.
  - Solve the unknown entailment:
    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge U(x_{1,2,3,4}, n_1) \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11})$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.
  $\Rightarrow$ refine in 2 phases.
- **Phase 1: preprocess the lemma template:**
  - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.
  - Solve the unknown entailment:
    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge U(x_{1,2,3,4}, n_1) \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11})$
  - Solution: $U(x_1, x_2, x_3, x_4, n_1) \equiv n_1 \geq 2$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:
  $\mathsf{dll}(x_{1,2,3,4}, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11} \wedge \textbf{?})$
- Adding Boolean constraints on only the consequent might *not be good enough* because the antecedent can be *too weak*.
  $\Rightarrow$ refine in 2 phases.
- **Phase 1: preprocess the lemma template:**
  - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.
  - Solve the unknown entailment:
    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge U(x_{1,2,3,4}, n_1) \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11})$
  - Solution: $U(x_1, x_2, x_3, x_4, n_1) \equiv n_1 \geq 2$
  - Obtain the preprocessed lemma template:
    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10,11})$

Step 2.2: Refine Lemma Templates' Consequents

- ▷ **Phase 2: fine-tune the preprocessed lemma template:**

Step 2.2: Refine Lemma Templates' Consequents

- **Phase 2: fine-tune the preprocessed lemma template:**
  - Find sufficient constraints containing all variables:

$$\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$$

Step 2.2: Refine Lemma Templates' Consequents

- **Phase 2: fine-tune the preprocessed lemma template:**
  - Find sufficient constraints containing all variables:

    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
  - Solve the unknown entailment:

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$

    $x_9 \mapsto x_{10}, x_{11} \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$

Step 2.2: Refine Lemma Templates' Consequents

- **Phase 2: fine-tune the preprocessed lemma template:**
  - Find sufficient constraints containing all variables:

    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
  - Solve the unknown entailment:

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$

    $x_9 \mapsto x_{10}, x_{11} \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$
  - Solution:

    $U(x_1, ..., x_{11}, n_{1,2}) \equiv x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9$

Step 2.2: Refine Lemma Templates' Consequents

- **Phase 2: fine-tune the preprocessed lemma template:**
  - Find sufficient constraints containing all variables:

    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
  - Solve the unknown entailment:

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$

    $x_9 \mapsto x_{10}, x_{11} \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$
  - Solution:

    $U(x_1, ..., x_{11}, n_{1,2}) \equiv x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9$
  - The discovered lemma:

    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$

    $x_9 \mapsto x_{10}, x_{11} \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9)$

Step 2.2: Refine Lemma Templates' Consequents

- **Phase 2: fine-tune the preprocessed lemma template:**
  - Find sufficient constraints containing all variables:
    $\mathsf{dll}(x_{1,2,3,4}, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_{5,6,7,8}, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
  - Solve the unknown entailment:
    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
    $x_9 \mapsto x_{10}, x_{11} \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$
  - Solution:
    $U(x_1, ..., x_{11}, n_{1,2}) \equiv x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9$
  - The discovered lemma:
    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
    $x_9 \mapsto x_{10}, x_{11} \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9)$
  - Normalize to obtain the motivating lemma L_3:
    $L_3 \triangleq \mathsf{dll}(a, b, c, d, m) \wedge m \geq 2 \vdash \exists w.(\mathsf{dll}(a, b, w, c, m-1) * c \mapsto w, d)$ (✓!!)

Proof System: General Inference Rules

$$\bot_{\mathsf{L}}^{\pi}\ \frac{}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \Pi_1 \rightarrow \mathsf{false}$$

$$\bot_{\mathsf{L}}^{\sigma}\ \frac{}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * u \overset{\iota_1}{\mapsto} \vec{t} * u \overset{\iota_2}{\mapsto} \vec{r} \wedge \Pi_1 \vdash F_2}$$

$$\vdash_{\pi}\ \frac{}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2$$

$$=_{\mathsf{L}}\ \frac{\mathcal{H},\ \mathcal{L},\ (\Sigma \wedge \Pi_1)[t/x] \vdash F_2[t/x]}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \wedge t{=}x \vdash F_2}$$

$$*\!\mapsto\ \frac{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2 \wedge u{=}w \wedge \vec{t}{=}\vec{r})}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * u \overset{\iota}{\mapsto} \vec{t} \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 * w \overset{\iota}{\mapsto} \vec{r} \wedge \Pi_2)}\ \vec{x} \# \mathtt{FV}(u, \vec{t})$$

$$\exists_{\mathsf{L}}\ \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2)$$

$$*\mathsf{P}\ \frac{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2 \wedge \vec{t}{=}\vec{r})}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{P}(\vec{r}) \wedge \Pi_2)}\ \vec{x} \# \mathtt{FV}(\vec{t})$$

$$\exists_{\mathsf{R}}\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.((\Sigma_2 \wedge \Pi_2)[t/y])}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}, y.(\Sigma_2 \wedge \Pi_2 \wedge t{=}y)}$$

$$\mathsf{P}_{\mathsf{L}}\ \frac{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \Sigma' \wedge \Pi_1 \wedge \Pi' \vdash F_2}\ \exists \vec{x}.(\Sigma' \wedge \Pi') \overset{\text{def}}{\Rightarrow} \mathsf{P}(\vec{t})$$

$$\mathsf{E}_{\mathsf{L}}\ \frac{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \mathsf{emp} \wedge \Pi_1 \vdash F_2}$$

$$\mathsf{P}_{\mathsf{R}}\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}, \vec{y}.(\Sigma_2 * \Sigma' \wedge \Pi_2 \wedge \Pi')}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{P}(\vec{t}) \wedge \Pi_2)}\ \exists \vec{y}.(\Sigma' \wedge \Pi') \overset{\text{def}}{\Rightarrow} \mathsf{P}(\vec{t})$$

$$\mathsf{E}_{\mathsf{R}}\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2)}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{emp} \wedge \Pi_2)}$$

($\mathcal{H}$ is a set of induction hypotheses, $\mathcal{L}$ is a set of lemmas)

Proof System: Induction and Lemma Rules

$$\mathsf{ID}\ \frac{\mathcal{H}',\ \mathcal{L},\ \Sigma_1 * \Sigma_1' \wedge \Pi_1 \wedge \Pi_1' \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ \mathcal{L},\ \Sigma_1 * \Sigma_n' \wedge \Pi_1 \wedge \Pi_n' \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger$: $\mathcal{H}' \triangleq \mathcal{H} \cup \{H\}$, where H is obtained by renaming vars of $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$, $\mathsf{P}(\vec{t}) \stackrel{\text{def}}{=} \exists \vec{x}_1.(\Sigma_1' \wedge \Pi_1') \vee \ldots \vee \exists \vec{x}_n.(\Sigma_n' \wedge \Pi_n')$, and $\mathtt{FV}(\vec{x}_1, \ldots, \vec{x}_n) \# \mathtt{FV}(\Sigma_1, \Pi_1)$.

$$\mathsf{IH}\ \frac{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4)\},\ \mathcal{L},\ \Sigma * \Sigma_4\theta \wedge \Pi_1 \wedge \Pi_4\theta \vdash F_2}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4)\},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger$: $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$, $\exists \theta, \Sigma.((\Sigma_1 * \mathsf{P}(\vec{t}) \ \cong\ \Sigma_3\theta * \mathsf{P}(\vec{r})\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta))$, and $\vec{x} \# \mathtt{FV}(\Sigma_1, \Pi_1)$

$$\mathsf{LM_L}\ \frac{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4)\},\ \Sigma * \Sigma_4\theta \wedge \Pi_1 \wedge \Pi_4\theta \vdash F_2}{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4)\},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger$: $\exists \theta, \Sigma.\ ((\Sigma_1 \cong \Sigma_3\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta))$, and $\vec{x} \# \mathtt{FV}(\Sigma_1, \Pi_1)$

$$\mathsf{LM_R}\ \frac{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash \exists \vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists \vec{x}.(\Sigma * \Sigma_3\theta \wedge \Pi_2 \wedge \Pi_3\theta)}{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash \exists \vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2)}\ \dagger$$

$\dagger$: $\exists \theta, \Sigma.((\Sigma_4\theta * \Sigma \cong \Sigma_2) \wedge (\vec{w}\theta \subseteq \vec{x}))$

Proof System: Synthesis Rules

$$\mathsf{U}^{\pi}_{\mathsf{L}}\,\frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_2\}}{\mathcal{H},\, \mathcal{L},\, \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2}$$

$$\mathsf{U}^{\sigma}_{\mathsf{L}}\,\frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\, \mathcal{L},\, \Sigma_1 \wedge \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2}\,\Sigma_1 \neq \mathsf{emp}$$

$$\mathsf{U}^{\pi}_{\mathsf{R}}\,\frac{\mathcal{A} \triangleq \{\Pi_1 \rightarrow \exists \vec{w}.(\Pi_2 \wedge U(\vec{x}))\}}{\mathcal{H},\, \mathcal{L},\, \Pi_1 \vdash \exists \vec{w}.(\Pi_2 \wedge U(\vec{x}))}$$

$$\mathsf{U}^{\sigma}_{\mathsf{R}}\,\frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\, \mathcal{L},\, \Pi_1 \wedge U(\vec{x}) \vdash \exists \vec{w}.(\Sigma_2 \wedge \Pi_2)}\,\Sigma_2 \neq \mathsf{emp}$$

$$\mathsf{U}_{\mathrm{IH}}\,\frac{\mathcal{H}',\, \mathcal{L},\, \Sigma * \Sigma_4\theta \wedge \Pi_1 \wedge \Pi_4\theta \vdash F_2 \qquad \mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow (\Pi_3 \wedge U(\vec{y}))\theta\}}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash \exists \vec{z}.(\Sigma_4 \wedge \Pi_4)\},\, \mathcal{L},\, \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \wedge U(\vec{x}) \vdash F_2}\,\dagger$$

$\dagger$: $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$, $\exists \theta, \Sigma.\, (\Sigma_1 * \mathsf{P}(\vec{t}) \cong \Sigma_3\theta * \mathsf{P}(\vec{r})\theta * \Sigma)$, $\vec{z} \# \mathtt{FV}(\Sigma_1, \Pi_1, \vec{x})$,

and $\mathcal{H}' = \mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash \exists \vec{z}.(\Sigma_4 \wedge \Pi_4)\}$

Implementation and Experiment

- Implementation:
  - SongbirdLS = Songbird + Lemma Synthesis.
  - Built on top of our existing prover Songbird [Ta et al., FM'16].
  - https://songbird-prover.github.io/lemma-synthesis
- Experiment:
  - Literature benchmarks: **sll_entl**, **slrd_entl** (SL-COMP 2014) and **slrd_indt** [Ta et al., FM'16].
  - New benchmark with more complex entailments and arithmetic constraints: **slrd_lm**.
  - Environment: Ubuntu 14.04, Intel E5-2620 (2.4GHz), RAM 64GB.
  - Date conducted: June 2017.

Experiment: Literature Benchmarks

	Benchmark		Proved Entailments						Average Time (s)						Lemma Stats.	
	Category		**Sld**	**Slk**	**Spn**	**Ccl**	**Sbd**	**Sls**	**Sld**	**Slk**	**Spn**	**Ccl**	**Sbd**	**Sls**	#Lemma	Avg Time
sll_entl	bolognesa	57	0	0	**57**	0	**57**	**57**	–	–	0.41	–	2.46	0.32	0	–
	clones	60	0	**60**	**60**	**60**	**60**	**60**	–	0.06	0.06	0.01	0.07	0.01	0	–
	smallfoot	54	0	**54**	**54**	**54**	**54**	**54**	–	0.05	0.05	0.22	0.06	0.08	0	–
slrd_entl	singly-ll	64	12	48	3	63	**64**	**64**	0.08	0.13	0.04	0.03	0.13	0.03	0	–
	doubly-ll	37	14	17	9	29	25	**35**	2.74	0.18	0.04	3.88	0.45	2.63	18[10]	2.8
	nested-ll	11	0	5	**11**	7	**11**	**11**	–	0.44	0.04	2.38	0.21	0.04	0	–
	skip-list	13	0	4	**13**	5	**13**	**13**	–	0.27	0.08	0.11	0.63	0.10	0	–
	tree	26	12	14	0	23	23	**24**	9.16	0.21	–	2.55	0.50	0.09	0	–
slrd_indt	ll/ll2	24	0	0	0	**24**	**24**	**24**	–	–	–	2.51	0.50	3.27	14[10]	0.9
	ll-even/odd	20	0	0	0	**20**	**20**	**20**	–	–	–	1.73	2.52	0.07	0	–
	ll-left/right	20	0	0	0	**20**	**20**	**20**	–	–	–	0.97	0.53	0.06	0	–
	misc.	32	0	0	0	31	**32**	**32**	–	–	–	8.20	1.72	0.61	2[2]	0.7
Total		418	38	202	207	336	403	**414**†	3.93	0.11	0.15	1.70	0.79	0.54	34[22]‡	1.9

Sld: Slide [Iosif et al., CADE'13] **Spn**: Spen [Enea et al., APLAS'14] **Ccl**: Cyclist [Brotherston et al., CADE'11]
Slk: Sleek [Chin et al., SCP'12] **Sbd**: Songbird [Ta et al., FM'16] **Sls**: SongbirdLS [this work]
BENCHMARKS: **sll_entl**, **slrd_entl**: SL-COMP 2014, **slrd_indt**: [Ta et al., FM'16]; Timeout 180 seconds
Sls proves 414†/418=99.04% benchmarks, synthesizes 34‡/418=0.08 lemmas per entailment

Experiment: New Benchmark slrd_lm (Synthetic)

Category		Proved Entails			Avg. Time (s)			Lemma Synthesis		Lemma Types		
		Ccl	Sbd	Sls	Ccl	Sbd	Sls	#Lemma	A.Time	#convert	#split	#combine
ll	20	17	18	**20**	26.27	4.84	10.25	28 [20]	0.3	18 (25)	0	2 (2)
ll-even/odd	17	**17**	**17**	**17**	0.21	0.03	0.02	0	–	0	0	0
ll+arith	72	0	10	**72**	–	0.04	2.82	81 [81]	0.6	0	81 (143)	0
ll-sorted	25	0	3	**19**	–	0.03	0.46	14 [14]	0.3	0	0	14 (30)
dll	22	**22**	**22**	**22**	0.38	0.80	6.81	12 [7]	4.9	7 (8)	0	0
dll+arith	94	0	17	**94**	–	1.63	26.39	236 [158]	8.1	74 (74)	69 (103)	15 (15)
ll/dll	5	**5**	**5**	**5**	0.07	0.03	0.03	0	–	0	0	0
ll/dll+arith	15	0	5	**15**	–	0.08	21.89	12 [10]	18.9	8 (8)	2 (2)	0
tree	18	7	7	**18**	0.18	0.04	7.50	18 [12]	0.2	8 (11)	0	4 (5)
tree+arith	12	0	3	**12**	–	1.15	42.81	20 [16]	10.6	8 (8)	4 (5)	4 (4)
Total	300	68	107	**294**†	6.76	1.29	13.69	421 [318]‡	5.9	123 (134)	156 (253)	39 (56)

Ccl, **Sbd**, **Sls** are the three best provers from the previous experiment.

†, ‡: **Sls** proves 98% of the benchmark and synthesizes 1.4 lemmas per entailment.

Proving Separation Logic Entailment by Learning from Failure

Overview of Existing Approaches

- The lemma synthesis framework is good at proving sophisticated entailments.

Overview of Existing Approaches

- The lemma synthesis framework is good at proving sophisticated entailments.
- However, it requires significant effort to handle unknown entailments. It also cannot synthesize lemma like $F \vdash$ false.

Overview of Existing Approaches

- The lemma synthesis framework is good at proving sophisticated entailments.
- However, it requires significant effort to handle unknown entailments. It also cannot synthesize lemma like $F \vdash$ false.
- We will consider simple entailments that existing induction proof systems and the lemma synthesis framework either cannot prove or prove with extra effort.

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 : \ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:
  - Rename variables in E_1 to record an IH:
    $H_1 : \ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:
  - Rename variables in E_1 to record an IH:
    $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - Unfold $\mathsf{ls}(x, y, n)$ to obtain a new entailment:
    $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:
  - Rename variables in E_1 to record an IH:
    $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - Unfold $\mathsf{ls}(x, y, n)$ to obtain a new entailment:
    $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - Match spatial formulas in H_1's and E_1''s antecedents by the substitution $\theta = [u/a, y/b, z/c, n{-}1/k]$

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:
  - Rename variables in E_1 to record an IH:
    $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - Unfold $\mathsf{ls}(x, y, n)$ to obtain a new entailment:
    $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - Match spatial formulas in H_1's and E_1''s antecedents by the substitution $\theta = [u/a, y/b, z/c, n{-}1/k]$
  - Apply θ on the pure formula $k{\geq}100$ of H_1 to obtain $n{-}1{\geq}100$.

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\text{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:
  - Rename variables in E_1 to record an IH:
    $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - Unfold $\mathsf{ls}(x, y, n)$ to obtain a new entailment:
    $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - Match spatial formulas in H_1's and E_1''s antecedents by the substitution $\theta = [u/a, y/b, z/c, n{-}1/k]$
  - Apply θ on the pure formula $k{\geq}100$ of H_1 to obtain $n{-}1{\geq}100$.
  - However, the pure formula $n{\geq}100$ of E_1' cannot prove that $n{-}1{\geq}100$.

Example 1: Proving a Simple Entailment

- Consider proving the following entailment:
  $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  with $\mathsf{ls}(x, y, n) \stackrel{\text{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1)$.
- Induction fails to prove E_1 due to the constraint $n{\geq}100$:
  - Rename variables in E_1 to record an IH:
    $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - Unfold $\mathsf{ls}(x, y, n)$ to obtain a new entailment:
    $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - Match spatial formulas in H_1's and E_1''s antecedents by the substitution $\theta = [u/a, y/b, z/c, n{-}1/k]$
  - Apply θ on the pure formula $k{\geq}100$ of H_1 to obtain $n{-}1{\geq}100$.
  - However, the pure formula $n{\geq}100$ of E_1' cannot prove that $n{-}1{\geq}100$.
- Although E_1 can be proved by the lemma synthesis framework, but it requires significant effort to handle unknown entailments.

Example 1: Proving a Simple Entailment

- Recall the failed induction proof attempt:
  - $E_1 : \ \mathsf{ls}(x, y, n) * y \mapsto z \wedge n \geq 100 \vdash \mathsf{ls}(x, z, n+1)$
  - $H_1 : \ \mathsf{ls}(a, b, k) * b \mapsto c \wedge k \geq 100 \vdash \mathsf{ls}(a, c, k+1)$
  - $E_1' : \ x \mapsto u * \mathsf{ls}(u, y, n-1) * y \mapsto z \wedge n \geq 100 \vdash \mathsf{ls}(x, z, n+1)$
  - $\theta = [u/a, y/b/z/c, n-1/k]$

Example 1: Proving a Simple Entailment

- Recall the failed induction proof attempt:
  - $E_1 : \ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $H_1 : \ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' : \ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta = [u/a, y/b/z/c, n{-}1/k]$
- *Observation:* the failure root cause is that the formula $n{\geq}100$ in E_1' cannot prove the constraint $n{-}1{\geq}100$ in $H_1\theta$.

Example 1: Proving a Simple Entailment

- Recall the failed induction proof attempt:
  - $E_1 : \ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $H_1 : \ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' : \ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta = [u/a, y/b/z/c, n{-}1/k]$
- *Observation:* the failure root cause is that the formula $n{\geq}100$ in E_1' cannot prove the constraint $n{-}1{\geq}100$ in $H_1\theta$.
- This formula $n{-}1{\geq}100$ corresponds to $k{\geq}100$ in H_1, which is obtained from renaming variables of $n{\geq}100$ in E_1.

Example 1: Proving a Simple Entailment

- Recall the failed induction proof attempt:
  - $E_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta = [u/a, y/b/z/c, n{-}1/k]$
- *Observation:* the failure root cause is that the formula $n{\geq}100$ in E_1' cannot prove the constraint $n{-}1{\geq}100$ in $H_1\theta$.
- This formula $n{-}1{\geq}100$ corresponds to $k{\geq}100$ in H_1, which is obtained from renaming variables of $n{\geq}100$ in E_1.
- *Our treatment:* removing the constraint $n{\geq}100$ to obtain weaker entailment $\overline{E}_1$, which can be proved by induction:
  - $\overline{E}_1 :\ \mathsf{ls}(x, y, n) * y{\mapsto}z \vdash \mathsf{ls}(x, z, n{+}1)$

Example 1: Proving a Simple Entailment

$$
\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash \exists v.\, z{=}v}\ \vdash_\pi}{\mathsf{true} \vdash \exists v.(\mathsf{ls}(v,z,0) \wedge z{=}v)}\ \mathsf{P_R}}{x{\mapsto}z \vdash \exists v.(x{\mapsto}v * \mathsf{ls}(v,z,0))}\ *{\mapsto}}{x{\mapsto}z \vdash \mathsf{ls}(x,z,1)}\ \mathsf{P_R}}{y{\mapsto}z \wedge x{=}y \wedge n{=}0 \vdash \mathsf{ls}(x,z,n{+}1)}\ =_{\mathsf{L}}
\qquad
\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{n{\geq}1 \vdash \exists v.(u{=}v \wedge n{\geq}0)}\ \vdash_\pi}{\mathsf{ls}(u,z,n) \wedge n{\geq}1 \vdash \exists v.(\mathsf{ls}(v,z,n) \wedge u{=}v \wedge n{\geq}0)}\ *\mathsf{P}}{\mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge n{\geq}1 \vdash \exists v.(\mathsf{ls}(v,z,n) \wedge u{=}v \wedge n{\geq}0)}\ \mathsf{IH}}{x{\mapsto}u * \mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge n{\geq}1 \vdash \exists v.(x{\mapsto}v * \mathsf{ls}(v,z,n) \wedge n{+}1{\geq}1)}\ *{\mapsto}}{x{\mapsto}u * \mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge n{\geq}1 \vdash \mathsf{ls}(x,z,n{+}1)}\ \mathsf{P_R}
$$

$$
\dfrac{\dfrac{\overline{E}_1 \triangleq \mathsf{ls}(x,y,n) * y{\mapsto}z \vdash \mathsf{ls}(x,z,n{+}1)}{E_1 \triangleq \mathsf{ls}(x,y,n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x,z,n{+}1)}\ W_\pi\text{: remove } n{\geq}100}{}\ \mathsf{ID}
$$

(Induction proof of the weaker entailment $\overline{E}_1$ and the original entailment E_1)

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  $E_2 : \mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  $E_2 : \mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  E_2 : $\mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - H_2 : $\mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$, by renaming variables of E_2.

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  E_2 : $\mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - H_2 : $\mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$, by renaming variables of E_2.
  - E_2' : $\mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$.

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  E_2 : $\mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash$ false
  with $\mathsf{lsr}(x, y, n) \stackrel{\text{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - H_2 : $\mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash$ false, by renaming variables of E_2.
  - E_2' : $\mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash$ false, by unfolding $\mathsf{lsr}(x, y, n)$.
  - $\theta = [x/a, u/b, z/c, n{-}1/k]$, by spatial-matching H_2's and E_2''s antecedents.

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  $E_2 : \mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - $H_2 :\ \mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$, by renaming variables of E_2.
  - $E_2' :\ \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$.
  - $\theta = [x/a, u/b, z/c, n{-}1/k]$, by spatial-matching H_2's and E_2''s antecedents.
  - Apply θ on the pure formula $a{\neq}b$ of H_2 to obtain $x{\neq}u$

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  $E_2 : \mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - $H_2 :\ \mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$, by renaming variables of E_2.
  - $E_2' :\ \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$.
  - $\theta = [x/a, u/b, z/c, n{-}1/k]$, by spatial-matching H_2's and E_2''s antecedents.
  - Apply θ on the pure formula $a{\neq}b$ of H_2 to obtain $x{\neq}u$
  - However, the pure formula $x{\neq}y$ of E_2' cannot prove that $x{\neq}u$.

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  E_2 : $\mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash$ false
  with $\mathsf{lsr}(x, y, n) \stackrel{\text{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - H_2 : $\mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash$ false, by renaming variables of E_2.
  - E_2' : $\mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash$ false, by unfolding $\mathsf{lsr}(x, y, n)$.
  - $\theta = [x/a, u/b, z/c, n{-}1/k]$, by spatial-matching H_2's and E_2''s antecedents.
  - Apply θ on the pure formula $a{\neq}b$ of H_2 to obtain $x{\neq}u$
  - However, the pure formula $x{\neq}y$ of E_2' cannot prove that $x{\neq}u$.
- The lemma synthesis framework cannot synthesize any lemma to prove an entailment like $F \vdash$ false.

Example 2: Entailment with Contradictory Consequent

- Consider proving the following entailment:
  $E_2 : \mathsf{lsr}(x, y, n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  with $\mathsf{lsr}(x, y, n) \stackrel{\mathsf{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists u.\, (\mathsf{ls}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)$
- Induction fails to prove E_2 due to the specific constraint $x{\neq}y$:
  - $H_2 :\ \mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$, by renaming variables of E_2.
  - $E_2' :\ \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$.
  - $\theta = [x/a, u/b, z/c, n{-}1/k]$, by spatial-matching H_2's and E_2''s antecedents.
  - Apply θ on the pure formula $a{\neq}b$ of H_2 to obtain $x{\neq}u$
  - However, the pure formula $x{\neq}y$ of E_2' cannot prove that $x{\neq}u$.
- The lemma synthesis framework cannot synthesize any lemma to prove an entailment like $F \vdash \mathsf{false}$.
- *Observation:* the failure root cause is that $x{\neq}y$ in E_2' cannot prove the constraint $x{\neq}u$ in the antecedent of $H_2\theta$.

Example 2: Entailment with Contradictory Consequent

- *Our treatment:* case-analyze the antecedent of E_2' on the unprovable constraint $x{\neq}u$ to derive provable entailments:
  - E_{21}' : $\mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge x{\neq}u \vdash$ false
  - E_{22}' : $\mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge x{=}u \vdash$ false

Example 2: Entailment with Contradictory Consequent

- *Our treatment:* case-analyze the antecedent of E_2' on the unprovable constraint $x{\neq}u$ to derive provable entailments:
  - E_{21}' : $\mathsf{lsr}(x,u,n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge x{\neq}u \vdash \mathsf{false}$
  - E_{22}' : $\mathsf{lsr}(x,u,n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge x{=}u \vdash \mathsf{false}$
- Induction proof of E_2 using the case analysis strategy:

$$
\dfrac{
\dfrac{}{x{\mapsto}z \wedge x{\neq}y \wedge x{=}y \wedge n{=}0 \vdash \mathsf{false}}\,\bot_{\mathsf{L}}^{\pi}
\qquad
\dfrac{
\dfrac{\dfrac{}{\mathsf{lsr}(x,x,n{-}1) * x{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \vdash \mathsf{false}}\,\bot_{\mathsf{L}}^{\sigma}}{E_{22}' : \mathsf{lsr}(x,u,n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \wedge x{=}u \vdash \mathsf{false}}\,=_{\mathsf{L}}
\quad
\dfrac{\dfrac{}{u{\mapsto}y \wedge x{\neq}y \wedge x{\neq}u \wedge n{\geq}1 \wedge \mathsf{false} \vdash \mathsf{false}}\,\bot_{\mathsf{L}}^{\pi}}{E_{21}' : \mathsf{lsr}(x,u,n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \wedge x{\neq}u \vdash \mathsf{false}}\,\mathsf{IH}
}{\mathsf{lsr}(x,u,n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \vdash \mathsf{false}}\,\mathsf{CA}
}{E_2 \triangleq \mathsf{lsr}(x,y,n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}}\,\mathsf{ID}
$$

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y \mapsto y$:

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  E_3 : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y \mapsto y$:
  - H_3 : $\mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$, by renaming variables of E_3.

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  E_3 : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y \mapsto y$:
  - H_3 : $\mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$, by renaming variables of E_3.
  - E_3' : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y \mapsto y$:
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$, by renaming variables of E_3.
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$
  - $\theta = [x/a, u/b, y/b, n-1/k]$, by spatial-matching H_3's and E_3''s antecedents.

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y{\mapsto}y$:
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b{\mapsto}b \vdash \mathsf{false}$, by renaming variables of E_3.
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * y{\mapsto}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$
  - $\theta = [x/a, u/b, y/b, n{-}1/k]$, by spatial-matching H_3's and E_3''s antecedents.
  - However, θ is an invalid substitution since b is mapped two both u and y.

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y{\mapsto}y$:
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b{\mapsto}b \vdash \mathsf{false}$, by renaming variables of E_3.
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * y{\mapsto}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$
  - $\theta = [x/a, u/b, y/b, n{-}1/k]$, by spatial-matching H_3's and E_3''s antecedents.
  - However, θ is an invalid substitution since b is mapped two both u and y.

- The lemma synthesis framework also cannot synthesize any lemmas for entailment like $F \vdash \mathsf{false}$.

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  E_3 : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y{\mapsto}y$:
  - H_3 : $\mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b{\mapsto}b \vdash \mathsf{false}$, by renaming variables of E_3.
  - E_3' : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * y{\mapsto}y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$
  - $\theta = [x/a, u/b, y/b, n{-}1/k]$, by spatial-matching H_3's and E_3''s antecedents.
  - However, θ is an invalid substitution since b is mapped two both u and y.

- The lemma synthesis framework also cannot synthesize any lemmas for entailment like $F \vdash \mathsf{false}$.

- *Observation:* The failure root cause is that b in θ is mapped two both u and y.

Example 3: Entailment with Contradictory Consequent

- Consider proving the following entailment:

  E_3 : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$

- Induction also fails to prove E_3 due to the singleton heap $y \mapsto y$:
  - H_3 : $\mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$, by renaming variables of E_3.
  - E_3' : $\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$, by unfolding $\mathsf{lsr}(x, y, n)$
  - $\theta = [x/a, u/b, y/b, n-1/k]$, by spatial-matching H_3's and E_3''s antecedents.
  - However, θ is an invalid substitution since b is mapped two both u and y.

- The lemma synthesis framework also cannot synthesize any lemmas for entailment like $F \vdash \mathsf{false}$.

- *Observation:* The failure root cause is that b in θ is mapped two both u and y.

- This b in H_3 corresponds to the variable y in E_3.

Example 3: Entailment with Contradictory Consequent

- *Our treatment:* generalize some instances of the variable y in E_3 by a fresh variable z to obtain a more general entailment $\overline{E}_3$:
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $\overline{E}_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto z \vdash \mathsf{false}$

Example 3: Entailment with Contradictory Consequent

- *Our treatment:* generalize some instances of the variable y in E_3 by a fresh variable z to obtain a more general entailment $\overline{E}_3$:
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}$
  - $\overline{E}_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}z \vdash \mathsf{false}$
- Induction proof of E_3 with the generalization strategy:

$$
\dfrac{
\dfrac{
\dfrac{
\dfrac{}{x{\mapsto}z \wedge x{=}\mathsf{nil} \wedge n{=}0 \wedge 100{=}0 \vdash \mathsf{false}}\,\bot_{\mathsf{L}}^{\pi}
\quad
\dfrac{}{x{\mapsto}u * \mathsf{ls}(u, \mathsf{nil}, 99) * x{\mapsto}z \wedge n{=}0 \vdash \mathsf{false}}\,\bot_{\mathsf{L}}^{\sigma}
}{\mathsf{ls}(x, \mathsf{nil}, 100) * x{\mapsto}z \wedge n{=}0 \vdash \mathsf{false}}\,\mathsf{ID}
}{\mathsf{ls}(x, \mathsf{nil}, 100) * y{\mapsto}z \wedge x{=}y \wedge n{=}0 \vdash \mathsf{false}}\,{=}_{\mathsf{L}}
\quad
\dfrac{
\dfrac{
\dfrac{}{y{\mapsto}z \wedge n{\geq}1 \wedge \mathsf{false} \vdash \mathsf{false}}\,\bot_{\mathsf{L}}^{\pi}
}{\mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * y{\mapsto}z \wedge n{\geq}1 \vdash \mathsf{false}}\,\mathsf{IH}
}{}
}{
\dfrac{\overline{E}_3 \triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}z \vdash \mathsf{false}}{E_3 \triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}}\,G_{\mathsf{v}}
}\,\mathsf{ID}
$$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:
  - Obtain: $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma * \mathsf{P}(\vec{r})\theta \wedge \Pi_1 \wedge \Pi_3\theta \vdash F_4\theta * \Sigma \wedge \Pi_1$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:
  - Obtain: $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma * \mathsf{P}(\vec{r})\theta \wedge \Pi_1 \wedge \Pi_3\theta \vdash F_4\theta * \Sigma \wedge \Pi_1$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:
  - Obtain: $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma * \mathsf{P}(\vec{r})\theta \wedge \Pi_1 \wedge \Pi_3\theta \vdash F_4\theta * \Sigma \wedge \Pi_1$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:
  - Obtain: $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma * \mathsf{P}(\vec{r})\theta \wedge \Pi_1 \wedge \Pi_3\theta \vdash F_4\theta * \Sigma \wedge \Pi_1$
  - Derive new sub-goal entailment: $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:
  - Obtain: $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma * \mathsf{P}(\vec{r})\theta \wedge \Pi_1 \wedge \Pi_3\theta \vdash F_4\theta * \Sigma \wedge \Pi_1$
  - Derive new sub-goal entailment: $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$

Why does an Induction Proof Fail?

- Induction hypothesis application in structural induction: applying an IH H to prove a goal entailment E:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$
  - $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$, with $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ (sub-structural relation).
- Need to match H's antecedent with E's antecedent:
  - *Spatial matching:* find a substitution θ and a spatial formula Σ s.t.: $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - *Pure matching:* prove that $\Pi_1 \rightarrow \Pi_3\theta$ is valid.
- If both the 2 matching steps succeed:
  - Obtain: $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma * \mathsf{P}(\vec{r})\theta \wedge \Pi_1 \wedge \Pi_3\theta \vdash F_4\theta * \Sigma \wedge \Pi_1$
  - Derive new sub-goal entailment: $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$
- If one of the two matching steps does not succeed, then the induction hypothesis application fails.

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.
- *Unmatchable variable:* a variable x is said to be *unmatchable* if it is mapped to two distinct terms t_1 and t_2, i.e., $t_1 \neq t_2$.

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.
- *Unmatchable variable:* a variable x is said to be *unmatchable* if it is mapped to two distinct terms t_1 and t_2, i.e., $t_1 \neq t_2$.
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [u/b, y/b, ...]$

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.
- *Unmatchable variable:* a variable x is said to be *unmatchable* if it is mapped to two distinct terms t_1 and t_2, i.e., $t_1 \neq t_2$.
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [u/b, y/b, ...] \Rightarrow$ the variable b is unmatchable.

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.
- *Unmatchable variable:* a variable x is said to be *unmatchable* if it is mapped to two distinct terms t_1 and t_2, i.e., $t_1 \neq t_2$.
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [u/b, y/b, ...] \Rightarrow$ the variable b is unmatchable.
- *Unmatchable constant:* a constant c is said to be *unmatchable* if it is mapped to a different term t, i.e., $(t \neq c)$.

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.
- *Unmatchable variable:* a variable x is said to be *unmatchable* if it is mapped to two distinct terms t_1 and t_2, i.e., $t_1 \neq t_2$.
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [u/b, y/b, ...] \Rightarrow$ the variable b is unmatchable.
- *Unmatchable constant:* a constant c is said to be *unmatchable* if it is mapped to a different term t, i.e., $(t \neq c)$.
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3'' : x \mapsto u * \mathsf{ls}(u, \mathsf{nil}, 99) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [99/100, ...]$

The Failure of the Spatial Matching

- Spatial matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Find θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$?
  - This spatial matching fails if θ contains *unmatchable* variables or constants.
- *Unmatchable variable:* a variable x is said to be *unmatchable* if it is mapped to two distinct terms t_1 and t_2, i.e., $t_1 \neq t_2$.
  - $E_3 : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3' : \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsr}(x, u, n-1) * u \mapsto y * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [u/b, y/b, ...] \Rightarrow$ the variable b is unmatchable.
- *Unmatchable constant:* a constant c is said to be *unmatchable* if it is mapped to a different term t, i.e., $(t \neq c)$.
  - $H_3 : \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsr}(a, b, k) * b \mapsto b \vdash \mathsf{false}$
  - $E_3'' : x \mapsto u * \mathsf{ls}(u, \mathsf{nil}, 99) * \mathsf{lsr}(x, y, n) * y \mapsto y \vdash \mathsf{false}$
  - $\theta = [99/100, ...] \Rightarrow$ the constant 100 is unmatchable.

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?
  - This pure matching fails if Π_3 contains an *unmatchable* pure formula.

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?
  - This pure matching fails if Π_3 contains an *unmatchable* pure formula.
- *Unmatchable pure formula:* Π_{u} is an unmatchable pure formula iff $\Pi_3 \equiv \Pi_{\mathsf{u}} \wedge \Pi$, for some Π, and $\Pi_1 \rightarrow \Pi\theta$ while $\Pi_1 \not\rightarrow \Pi_{\mathsf{u}}\theta$.

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?
  - This pure matching fails if Π_3 contains an *unmatchable* pure formula.
- *Unmatchable pure formula:* Π_{u} is an unmatchable pure formula iff $\Pi_3 \equiv \Pi_{\mathsf{u}} \wedge \Pi$, for some Π, and $\Pi_1 \rightarrow \Pi\theta$ while $\Pi_1 \not\rightarrow \Pi_{\mathsf{u}}\theta$.
  - $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta_1 = [n{-}1/k, ...]$

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?
  - This pure matching fails if Π_3 contains an *unmatchable* pure formula.
- *Unmatchable pure formula:* Π_{u} is an unmatchable pure formula iff $\Pi_3 \equiv \Pi_{\mathsf{u}} \wedge \Pi$, for some Π, and $\Pi_1 \rightarrow \Pi\theta$ while $\Pi_1 \not\rightarrow \Pi_{\mathsf{u}}\theta$.
  - $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta_1 = [n{-}1/k, ...]$

    $\Rightarrow k{\geq}100$ is unmatchable since $n{\geq}100 \not\rightarrow n{-}1{\geq}100 \quad (\equiv (k{\geq}100)\theta_1)$.

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?
  - This pure matching fails if Π_3 contains an *unmatchable* pure formula.
- *Unmatchable pure formula:* Π_{u} is an unmatchable pure formula iff $\Pi_3 \equiv \Pi_{\mathsf{u}} \wedge \Pi$, for some Π, and $\Pi_1 \rightarrow \Pi\theta$ while $\Pi_1 \not\rightarrow \Pi_{\mathsf{u}}\theta$.
  - $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta_1 = [n{-}1/k, ...]$
    $\Rightarrow k{\geq}100$ is unmatchable since $n{\geq}100 \not\rightarrow n{-}1{\geq}100 \quad (\equiv (k{\geq}100)\theta_1)$.
  - $H_2 :\ \mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$
  - $E_2' :\ \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  - $\theta_2 = [x/a, u/b, ...]$

The Failure of the Pure Matching

- Pure matching:
  - $E \triangleq \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$ and $H \triangleq \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3$
  - Found θ and Σ such that $\mathsf{P}(\vec{t}) \cong \mathsf{P}(\vec{r})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$
  - Prove if $\Pi_1 \rightarrow \Pi_3\theta$ is valid?
  - This pure matching fails if Π_3 contains an *unmatchable* pure formula.
- *Unmatchable pure formula:* Π_{u} is an unmatchable pure formula iff $\Pi_3 \equiv \Pi_{\mathsf{u}} \wedge \Pi$, for some Π, and $\Pi_1 \rightarrow \Pi\theta$ while $\Pi_1 \not\rightarrow \Pi_{\mathsf{u}}\theta$.
  - $H_1 :\ \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1)$
  - $E_1' :\ x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$
  - $\theta_1 = [n{-}1/k, ...]$
    $\Rightarrow k{\geq}100$ is unmatchable since $n{\geq}100 \not\rightarrow n{-}1{\geq}100 \quad (\equiv (k{\geq}100)\theta_1)$.
  - $H_2 :\ \mathsf{lsr}(a, b, k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}$
  - $E_2' :\ \mathsf{lsr}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}$
  - $\theta_2 = [x/a, u/b, ...]$
    $\Rightarrow a{\neq}b$ is unmatchable since $x{\neq}y \not\rightarrow x{\neq}u \quad (\equiv (a{\neq}b)\theta_2)$.

Our Approach to Learn from the Failure

- Given an unsuccessful induction proof attempt...

Our Approach to Learn from the Failure

- ▷ Given an unsuccessful induction proof attempt...
- ▷ Step 1: Identify the failure root cause of the induction hypothesis application.

Our Approach to Learn from the Failure

- Given an unsuccessful induction proof attempt...
- Step 1: Identify the failure root cause of the induction hypothesis application.
- Step 2: If it is due to *unmatchable* constants/variables, then *generalize* them by fresh variables to obtain a more general IH.

Our Approach to Learn from the Failure

- ▷ Given an unsuccessful induction proof attempt...
- ▷ Step 1: Identify the failure root cause of the induction hypothesis application.
- ▷ Step 2: If it is due to *unmatchable* constants/variables, then *generalize* them by fresh variables to obtain a more general IH.
- ▷ Step 3: If it is due to *unmatchable* pure formula, then perform one of the following two solutions:

Our Approach to Learn from the Failure

- Given an unsuccessful induction proof attempt...
- Step 1: Identify the failure root cause of the induction hypothesis application.
- Step 2: If it is due to *unmatchable* constants/variables, then *generalize* them by fresh variables to obtain a more general IH.
- Step 3: If it is due to *unmatchable* pure formula, then perform one of the following two solutions:
  - *Weaken* the unmatchable pure formula from the original goal entailment to obtain a more general IH.

Our Approach to Learn from the Failure

- Given an unsuccessful induction proof attempt...
- Step 1: Identify the failure root cause of the induction hypothesis application.
- Step 2: If it is due to *unmatchable* constants/variables, then *generalize* them by fresh variables to obtain a more general IH.
- Step 3: If it is due to *unmatchable* pure formula, then perform one of the following two solutions:
  - *Weaken* the unmatchable pure formula from the original goal entailment to obtain a more general IH.
  - *Case-analyze* the current entailment on the unmatchable pure formula to apply the existing IH.

Proof System: General Inference Rules

$$\bot_{\mathsf{L}}^{\pi}\ \frac{}{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \Pi_1 \rightarrow \mathsf{false}$$

$$\bot_{\mathsf{L}}^{\sigma}\ \frac{}{\mathcal{H},\ \Sigma_1 * u \overset{\iota_1}{\mapsto} \vec{t} * u \overset{\iota_2}{\mapsto} \vec{r} \wedge \Pi_1 \vdash F_2}$$

$$\vdash_{\pi}\ \frac{}{\mathcal{H},\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2$$

$$=_{\mathsf{L}}\ \frac{\mathcal{H},\ (\Sigma \wedge \Pi_1)[t/x] \vdash F_2[t/x]}{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \wedge t{=}x \vdash F_2}$$

$$*{\mapsto}\ \frac{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2 \wedge u{=}w \wedge \vec{t}{=}\vec{r})}{\mathcal{H},\ \Sigma_1 * u \overset{\iota}{\mapsto} \vec{t} \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 * w \overset{\iota}{\mapsto} \vec{r} \wedge \Pi_2)}\ \vec{x} \# \mathtt{FV}(u, \vec{t})$$

$$\exists_{\mathsf{L}}\ \frac{\mathcal{H},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2)$$

$$*\mathsf{P}\ \frac{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2 \wedge \vec{t}{=}\vec{r})}{\mathcal{H},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{P}(\vec{r}) \wedge \Pi_2)}\ \vec{x} \# \mathtt{FV}(\vec{t})$$

$$\exists_{\mathsf{R}}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}.((\Sigma_2 \wedge \Pi_2)[t/y])}{\mathcal{H},\ F_1 \vdash \exists \vec{x}, y.(\Sigma_2 \wedge \Pi_2 \wedge t{=}y)}$$

$$\mathsf{P}_{\mathsf{L}}\ \frac{\mathcal{H},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \Sigma_1 * \Sigma' \wedge \Pi_1 \wedge \Pi' \vdash F_2}\ \exists \vec{x}.(\Sigma' \wedge \Pi') \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{t})$$

$$\mathsf{E}_{\mathsf{L}}\ \frac{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \Sigma_1 * \mathsf{emp} \wedge \Pi_1 \vdash F_2}$$

$$\mathsf{P}_{\mathsf{R}}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}, \vec{y}.(\Sigma_2 * \Sigma' \wedge \Pi_2 \wedge \Pi')}{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{P}(\vec{t}) \wedge \Pi_2)}\ \exists \vec{y}.(\Sigma' \wedge \Pi') \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{t})$$

$$\mathsf{E}_{\mathsf{R}}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2)}{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 * \mathsf{emp} \wedge \Pi_2)}$$

($\mathcal{H}$ is a set of induction hypotheses)

Proof System: Induction Rules

$$\text{ID}\ \frac{\mathcal{H}',\ \Sigma_1 * \Sigma_1' \wedge \Pi_1 \wedge \Pi_1' \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ \Sigma_1 * \Sigma_n' \wedge \Pi_1 \wedge \Pi_n' \vdash F_2}{\mathcal{H},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger:\ \mathcal{H}' \triangleq \mathcal{H} \cup \{H\}$, where H is obtained by renaming variables of $\Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2$, and $\mathsf{P}(\vec{t}) \stackrel{\text{def}}{=} \exists \vec{x}_1.(\Sigma_1' \wedge \Pi_1')\ \vee\ \ldots\ \vee\ \exists \vec{x}_n.(\Sigma_n' \wedge \Pi_n')$, and $\mathtt{FV}(\vec{x}_1, ..., \vec{x}_n) \# \mathtt{FV}(\Sigma_1, \Pi_1)$.

$$\text{IH}\ \frac{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4)\},\ \Sigma * \Sigma_4\theta \wedge \Pi_1 \wedge \Pi_4\theta \vdash F_2}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash \exists \vec{x}.(\Sigma_4 \wedge \Pi_4)\},\ \Sigma_1 * \mathsf{P}(\vec{t}) \wedge \Pi_1 \vdash F_2}\ \mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r}); \dagger$$

$\dagger: \exists \theta, \Sigma.((\Sigma_1 * \mathsf{P}(\vec{t}) \cong \Sigma_3\theta * \mathsf{P}(\vec{r})\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta))$, and $\vec{x} \# \mathtt{FV}(\Sigma_1, \Pi_1)$

($\mathcal{H}$ is a set of induction hypotheses)

Proof System: Learning from Failure Rules

$$G_{\mathsf{v}}\ \frac{\mathcal{H},\ F_1 \vdash F_2}{\mathcal{H},\ F_1[x/u] \vdash F_2}\, u \notin \mathtt{FV}(F_2) \qquad G_{\mathsf{c}}\ \frac{\mathcal{H},\ F_1 \vdash F_2}{\mathcal{H},\ F_1[c/u] \vdash F_2}\, u \notin \mathtt{FV}(F_2)$$

$$W_{\pi}\ \frac{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \wedge \Pi \vdash F_2} \qquad \mathsf{CA}\ \frac{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \wedge \Pi \vdash F_2 \qquad \mathcal{H},\ \Sigma_1 \wedge \Pi_1 \wedge \neg\Pi \vdash F_2}{\mathcal{H},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}$$

Where x, c, Π are the *unmatchable* variable, constant, pure formula.
They are identified from the unsuccessful spatial matching and pure matching.

Implementation and Experiment

- Implementation:
  - SongbirdFL = Songbird + Failure Learning.
  - Built on top of our existing prover Songbird [Ta et al., FM'16].
  - https://songbird-prover.github.io
- Experiment:
  - Literature benchmarks: **sll_entl**, **slrd_entl** (SL-COMP-2014) and **slrd_indt, slrd_lm** [Ta et al., FM'16, Ta et al., POPL'18].
  - New benchmark of entailments with contradictory consequents: **slrd_unsat**.
  - Environment: Ubuntu 14.04, Intel E5-2620 (2.4GHz), RAM 64GB.
  - Date conducted: December 2017.

Experiment with Benchmarks from SL-COMP 2014

Benchmark			Proved Entails				Average Time (s)				Sfl Stats.			
	Category		**Ccl**	**Sbd**	**Sls**	**Sfl**	**Ccl**	**Sbd**	**Sls**	**Sfl**	CA	W_π	G_v	G_c
sll_entl	bolognesa	57	0	**57**	**57**	**57**	–	0.79	0.79	0.49	0	0	0	0
	clones	60	**60**	**60**	**60**	**60**	0.01	0.03	0.03	0.01	0	0	0	0
	smallfoot	55	**55**	**55**	**55**	**55**	0.20	0.13	0.13	0.07	0	0	0	0
slrd_entl	singly-ll	64	63	63	**64**	**64**	0.04	0.07	0.06	0.03	1	1	0	0
	doubly-ll	37	29	25	**35**	27	3.75	0.09	4.06	0.11	0	1	0	1
	nested-ll	11	7	**11**	**11**	**11**	1.96	0.10	0.09	0.04	0	0	0	0
	skip-list	13	5	**13**	**13**	**13**	0.09	0.23	0.24	0.18	0	0	0	0
	tree	26	23	**24**	**24**	**24**	0.91	4.30	0.20	0.09	0	0	0	0
Total		323	242	308	**319**	311	0.65	0.55	0.66	**0.14**†	1	2	0	1

Ccl: Cyclist [Brotherston et al., CADE'11], **Sbd**: Songbird [Ta et al., FM'16]

Sls: SongbirdLS [Ta et al., POPL'18], **Sfl**: SongbirdFL [this work]

†: **Sfl** is the fastest prover (Timeout 120 seconds).

Experiment with other Literature Benchmarks

Benchmark			Proved Entails				Average Time (s)				Sfl Stats.			
	Category		Ccl	Sbd	Sls	Sfl	Ccl	Sbd	Sls	Sfl	CA	W_π	G_v	G_c
slrd_indt	ll/ll2	24	**24**	**24**	**24**	21	0.38	0.14	4.02	0.12	0	0	0	0
	ll-even/odd	20	**20**	**20**	**20**	**20**	1.35	0.11	0.18	0.05	0	0	0	0
	ll-left/right	20	**20**	**20**	**20**	**20**	1.04	0.09	0.16	0.04	0	0	0	0
	misc.	32	31	**32**	**32**	**32**	5.90	0.19	0.84	0.18	0	0	0	0
slrd_lm	ll/rev	20	17	18	**20**	**20**	23.42	3.71	10.92	0.38	0	0	0	2
	ll-even/odd	17	**17**	**17**	**17**	**17**	0.23	0.05	0.06	0.02	0	0	0	0
	ll/rev+arith†	72	0	11	**72**	21	–	1.95	3.79	0.82	5	5	0	9
	ll-sorted†	25	0	7	**19**	17	–	0.60	1.12	0.09	1	1	0	0
	dll/rev	22	**22**	**22**	**22**	**22**	0.35	0.54	7.84	0.03	0	0	0	1
	dll/rev+arith†	94	0	17	**94**	84	–	0.61	37.20	2.12	37	62	0	34
	ll/dll	5	**5**	**5**	**5**	**5**	0.04	0.06	0.07	0.02	0	0	0	0
	ll/dll+arith†	15	0	5	**15**	14	–	0.19	23.71	2.03	4	5	0	4
	tree	18	7	14	**18**	13	0.13	12.81	9.07	3.91	0	0	0	0
	tree+arith†	12	0	3	**12**	5	–	1.00	53.04	17.84	2	9	0	0
Total		396	163	215	**390**	311	3.99	1.46	14.02	**1.23**†	49	82	0	50

Ccl: Cyclist [Brotherston et al., CADE'11], **Sbd**: Songbird [Ta et al., FM'16]
Sls: SongbirdLS [Ta et al., POPL'18], **Sfl**: SongbirdFL [this work]
Benchmarks from [Ta et al., FM'16, Ta et al., POPL'18]; Timeout 120 seconds

Experiment with Benchmark slrd_unsat

Benchmark		Proved Entails					Average Time (s)					Sfl Stats.			
Category		**Csa**	**Ccl**	**Sbd**	**Sls**	**Sfl**	**Csa**	**Ccl**	**Sbd**	**Sls**	**Sfl**	CA	W_π	G_v	G_c
ll/rev	80	**80**	60	25	54	77	0.01	0.04	0.04	0.30	0.31	68	5	15	12
ll/rev+arith†	250	0	0	86	121	**245**	–	–	5.73	3.70	1.22	272	150	14	33
dll/rev	42	**42**	36	12	34	**42**	0.01	0.05	0.05	0.09	0.76	51	8	0	0
dll/rev+arith†	146	0	0	61	97	**146**	–	–	0.15	1.09	1.15	168	95	0	0
tree/rev	54	**54**	33	23	28	46	0.01	0.05	0.04	0.37	0.91	54	0	13	5
tree/rev+arith†	260	0	0	178	179	**248**	–	–	0.07	1.00	1.38	252	0	39	33
Total	832	176	129	385	513	**804**†	0.01	0.04	1.34	1.48	1.13	865	258	81	83

Csa: CyclistSAT [Brotherston et al., LICS'14], **Ccl**: Cyclist [Brotherston et al., CADE'11]

Sbd: Songbird [Ta et al., FM'16], **Sls**: SongbirdLS [Ta et al., POPL'18], **Sfl**: SongbirdFL [this work]

Benchmark: slrd_unsat (synthetic); Timeout 120 seconds; †: **Sfl** is the best prover.

Discussions, Future Works, and Conclusions

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction $=$ Mutual Induction $<$ Lemma Synthesis.

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction $=$ Mutual Induction $<$ Lemma Synthesis.
  - Standard Induction $=$ Mutual Induction $<$ Learning from Failure.

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction = Mutual Induction < Lemma Synthesis.
  - Standard Induction = Mutual Induction < Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure.

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction $=$ Mutual Induction $<$ Lemma Synthesis.
  - Standard Induction $=$ Mutual Induction $<$ Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure. 😅

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction $=$ Mutual Induction $<$ Lemma Synthesis.
  - Standard Induction $=$ Mutual Induction $<$ Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure. 😅
- Comparison on the effectiveness:

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction = Mutual Induction < Lemma Synthesis.
  - Standard Induction = Mutual Induction < Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure. 😅
- Comparison on the effectiveness:
  - The mutual induction is good at proving simple entailments. (✓)

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction = Mutual Induction < Lemma Synthesis.
  - Standard Induction = Mutual Induction < Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure. 😅
- Comparison on the effectiveness:
  - The mutual induction is good at proving simple entailments. (✓)
  - The lemma synthesis is useful for sophisticated entailments. (✓)

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction = Mutual Induction < Lemma Synthesis.
  - Standard Induction = Mutual Induction < Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure. 😅
- Comparison on the effectiveness:
  - The mutual induction is good at proving simple entailments. (✓)
  - The lemma synthesis is useful for sophisticated entailments. (✓)
  - The learning from failure framework is better in proving entailments with specific constraints, especially with contradictory consequents. (✓)

Discussions

- We have presented three entailment proof systems:
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- They are sound but incomplete, since they are based on induction.
- Comparison on the completeness:
  - Standard Induction = Mutual Induction < Lemma Synthesis.
  - Standard Induction = Mutual Induction < Learning from Failure.
  - Lemma Synthesis ...???... Learning from Failure. 😅
- Comparison on the effectiveness:
  - The mutual induction is good at proving simple entailments. (✓)
  - The lemma synthesis is useful for sophisticated entailments. (✓)
  - The learning from failure framework is better in proving entailments with specific constraints, especially with contradictory consequents. (✓)

    ⇒ Combining these systems? YAY!!

Related Publications

- *Automated Mutual Explicit Induction Proof in Separation Logic*
  Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
  International Symposium on Formal Methods (FM) 2016

Related Publications

- *Automated Mutual Explicit Induction Proof in Separation Logic*
  Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
  International Symposium on Formal Methods (FM) 2016
- *Automated Lemma Synthesis in Symbolic-Heap Separation Logic*
  Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
  Symposium on Principles of Programming Languages (POPL) 2018

Related Publications

- *Automated Mutual Explicit Induction Proof in Separation Logic*
  Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
  International Symposium on Formal Methods (FM) 2016
- *Automated Lemma Synthesis in Symbolic-Heap Separation Logic*
  Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
  Symposium on Principles of Programming Languages (POPL) 2018
- *Automated Induction in Separation Logic: Learning From Failure*
  Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
  NUS - Technical Report 2018

Future Works

- How to improve the completeness of the entailment proof: restrict the logic fragment or propose new proof techniques?

Future Works

- How to improve the completeness of the entailment proof: restrict the logic fragment or propose new proof techniques?
- How to effectively implement an inductive prover: manually design the proof search heuristics or utilize machine learning to automatically learn the optimized heuristics?

Future Works

- How to improve the completeness of the entailment proof: restrict the logic fragment or propose new proof techniques?
- How to effectively implement an inductive prover: manually design the proof search heuristics or utilize machine learning to automatically learn the optimized heuristics?
- More general problems: handling magical wand, supporting frame inference and bi-abduction inference, applying to verify real-world programs.

Conclusion

- ▷ Theoretical contributions: induction-based proof systems for separation logic entailments
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.

Conclusion

- Theoretical contributions: induction-based proof systems for separation logic entailments
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- Practical contribution: the inductive prover Songbird

  - https://songbird-prover.github.io
  - Written in OCaml, using Emacs, 25k LoC.
  - Outperforms all existing separation logic provers.

Conclusion

- Theoretical contributions: induction-based proof systems for separation logic entailments
  - The mutual induction proof system.
  - The lemma synthesis framework.
  - The learning from failure framework.
- Practical contribution: the inductive prover Songbird
  - https://songbird-prover.github.io
  - Written in OCaml, using Emacs, 25k LoC.
  - Outperforms all existing separation logic provers.

Backup slides

Mutual Induction: The Proof Search Procedure

Procedure Prove$(\mathcal{H}, \rho, F \vdash G)$

Input: $\mathcal{H}$ and ρ are a hypothesis set and the proof trace.
Output: Validity result, derived entailments, used hypotheses.

$\overline{\mathcal{R}} \leftarrow \{\, \bar{\mathsf{R}} \mid \bar{\mathsf{R}} = \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \rho, F \vdash G)) \ \wedge \ \mathsf{R} \in \mathcal{R} \,\}$
if $\overline{\mathcal{R}} = \varnothing$ **then return** $(\texttt{Unkn}, \varnothing, \varnothing)$ *// no rule is selected*
 for each $\bar{\mathsf{R}}$ **in** $\overline{\mathcal{R}}$ **do**
  if $\bar{\mathsf{R}} \in \{\, \vdash_{\pi}, \bot_{\mathsf{L}}^{\pi}, \bot_{\mathsf{L}}^{\sigma} \,\}$ **then return** $(\texttt{Valid}, \varnothing, \varnothing)$ *// R is an axiom rule*
  $r \leftarrow \texttt{Valid}$; $\mathcal{H}_{\text{used}} \leftarrow \varnothing$; $\mathcal{H}_{\text{drv}} \leftarrow \varnothing$
  if $\bar{\mathsf{R}} = \mathsf{IH}$ with hypothesis E **then** $\mathcal{H}_{\text{used}} \leftarrow \mathcal{H}_{\text{used}} \cup \{E\}$
  for each $(\mathcal{H}_i, \rho_i, F_i \vdash G_i)$ **in** $\mathsf{GetPremises}(\bar{\mathsf{R}})$ **do**
   $r, \mathcal{H}_{\text{drv}}, \mathcal{H}'_{\text{used}} \leftarrow \mathsf{Prove}(\mathcal{H}_i \oplus \mathcal{H}_{\text{drv}}, \rho_i, F_i \vdash G_i)$
   if $r = \texttt{Unkn}$ **then break**
   $\mathcal{H}_{\text{used}} \leftarrow \mathcal{H}_{\text{used}} \cup \mathcal{H}'_{\text{used}}$
  if $r = \texttt{Unkn}$ **then continue**
  if $\mathcal{H}_{\text{used}} \subseteq (\mathcal{H}_{\text{drv}} \cup \{F \vdash G\})$ **then** $\mathcal{H}_{\text{drv}} \leftarrow \mathcal{H}_{\text{drv}} \oplus \{(F \vdash G, \checkmark)\}$
  else $\mathcal{H}_{\text{drv}} \leftarrow \mathcal{H}_{\text{drv}} \oplus \{(F \vdash G, ?)\}$
  return $(\texttt{Valid}, \mathcal{H}_{\text{drv}}, \mathcal{H}_{\text{used}})$ *// all derived premises are proved*
return $(\texttt{Unkn}, \varnothing, \varnothing)$ *// all rules fail to prove $F \vdash G$*

Lemma Synthesis: The Proof Search Procedure

Procedure Prove$(\mathcal{H}, \mathcal{L}, F_1 \vdash F_2, mode)$

$\overline{\mathcal{R}} \leftarrow \{\, \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \mathcal{L}, F_1 \vdash F_2)) \mid \mathsf{R} \in \mathcal{R} \,\}$
$\mathcal{L}_{\text{syn}} \leftarrow \varnothing$
if $(mode = \textsc{SynLM})$ **and** $\mathsf{NeedLemmas}(F_1 \vdash F_2, \overline{\mathcal{R}})$ **then**
  $\mathcal{L}_{\text{syn}} \leftarrow \mathsf{SynthesizeLemma}(\mathcal{L},\ F_1 \vdash F_2)$ *// Synthesize new lemmas*
  $\overline{\mathcal{R}} \leftarrow \overline{\mathcal{R}} \cup \{\, \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \mathcal{L}_{\text{syn}}, F_1 \vdash F_2)) \mid \mathsf{R} \in \{\mathsf{LM}_{\mathrm{L}}, \mathsf{LM}_{\mathrm{R}}\} \,\}$
for each $\bar{\mathsf{R}}$ **in** $\overline{\mathcal{R}}$ **do**
  $\mathcal{A} \leftarrow \varnothing$
  if $\mathsf{IsSynthesiRule}(\bar{\mathsf{R}})$ **then** *// $\bar{\mathsf{R}} \in \{\, \mathsf{U}_L^{\pi}, \mathsf{U}_R^{\pi}, \mathsf{U}_L^{\sigma}, \mathsf{U}_R^{\sigma}, \mathsf{U}_{\mathrm{IH}} \,\}$*
    $\mathcal{A} \leftarrow \mathsf{Assumptions}(\bar{\mathsf{R}})$
  if $\mathsf{IsAxiomRule}(\bar{\mathsf{R}})$ **then**
    $\xi \leftarrow \mathsf{CreateWitnessProofTree}(F_1 \vdash F_2, \bar{\mathsf{R}}, \varnothing)$
    return $(\textsc{Valid}\langle \xi \rangle, \mathcal{L}_{\text{syn}}, \mathcal{A})$
  $(\mathcal{H}_i, \mathcal{L}_i, F_{1i} \vdash F_{2i})_{i=1\ldots n} \leftarrow \mathsf{Premises}(\bar{\mathsf{R}})$
  $(r_i, \mathcal{L}_{\text{syn}}^i, \mathcal{A}_i)_{i=1\ldots n} \leftarrow \mathsf{Prove}(\mathcal{H}_i,\ \mathcal{L} \cup \mathcal{L}_{\text{syn}} \cup \mathcal{L}_i,\ F_{1i} \vdash F_{2i}, mode)_{i=1\ldots n}$
  if $r_i = \textsc{Valid}\langle \xi_i \rangle$ **for all** $i = 1 \ldots n$ **then**
    $\xi \leftarrow \mathsf{CreateWitnessProofTree}(F_1 \vdash F_2, \bar{\mathsf{R}}, \{\, \xi_1, \ldots, \xi_n \,\})$
    return $(\textsc{Valid}\langle \xi \rangle,\ \mathcal{L}_{\text{syn}} \cup \mathcal{L}_{\text{syn}}^1 \cup \ldots \cup \mathcal{L}_{\text{syn}}^n,\ \mathcal{A} \cup \mathcal{A}_1 \cup \ldots \cup \mathcal{A}_n)$
return $(\texttt{Unkn}, \varnothing, \varnothing)$

Learning From Failure: The Spatial Matching Algorithm

Procedure MatchSpatial$(\Sigma_{\mathsf{h}}, \Sigma_{\mathsf{e}}, \mathrm{M})$

Input: $\Sigma_{\mathsf{h}}, \Sigma_{\mathsf{e}}$ are two heap formulas, M is a set of mappings.
Output: A set of both discovered spatial matchers and failed local mappings

$\mathcal{U} \leftarrow \varnothing$
if $\Sigma_{\mathsf{h}} \not\equiv \mathsf{emp}$ **then**
  $\sigma_{\mathsf{h}}, \Sigma'_{\mathsf{h}} \leftarrow$ SplitOneSpatialAtom(Σ_{h}) // $\sigma_{\mathsf{h}} * \Sigma'_{\mathsf{h}} \cong \Sigma_{\mathsf{h}}$
  $\mathcal{A} \leftarrow \{(\sigma_{\mathsf{e}}, \Sigma'_{\mathsf{e}}) \mid \sigma_{\mathsf{e}} \in \mathsf{SpatialAtoms}(\Sigma_{\mathsf{e}}) \wedge \mathsf{SamePredSymbol}(\sigma_{\mathsf{e}}, \sigma_{\mathsf{h}}) \wedge \sigma_{\mathsf{e}} * \Sigma'_{\mathsf{e}} \cong \Sigma_{\mathsf{e}}\}$
  for each $(\sigma_{\mathsf{e}}, \Sigma'_{\mathsf{e}})$ **in** $\mathcal{A}$ **do**
    $\mathrm{M}' \leftarrow \mathrm{M}$
    for each (t_h, t_e) **in** ParameterPairs$(\sigma_{\mathsf{h}}, \sigma_{\mathsf{e}})$ **do**
      $\mathrm{M}' \leftarrow$ MatchTerm(t_h, t_e, M')
    $\mathcal{U} \leftarrow \mathcal{U} \cup$ MatchSpatial$(\Sigma'_{\mathsf{h}}, \Sigma'_{\mathsf{e}}, \mathrm{M}')$
else // $\Sigma_{\mathsf{h}} \equiv \mathsf{emp}$
  for each m **in** M **do**
    if $\exists\, \mathtt{Const}\langle c, _, t\rangle \in m$ **where** $c \neq t$ **then**
      $\mathcal{U} \leftarrow \mathcal{U} \cup \{\mathtt{Fail}\langle m\rangle\}$
    else if $\exists\, \mathtt{Var}\langle x, _, t\rangle, \mathtt{Var}\langle x, _, t'\rangle \in m$ **where** $t \neq t'$ **then**
      $\mathcal{U} \leftarrow \mathcal{U} \cup \{\mathtt{Fail}\langle m\rangle\}$
    else
      $\theta \leftarrow [t/x \mid \mathtt{Var}\langle x, _, t\rangle \in m]$ // *get all substitutions from* m
      $\mathcal{U} \leftarrow \mathcal{U} \cup \{\mathtt{Success}\langle \theta, \Sigma_{\mathsf{e}}\rangle\}$
return $\mathcal{U}$

Learning From Failure: The Pure Matching Algorithm

Procedure MatchPure($\Pi_\mathsf{h}, \Pi_\mathsf{e}, \theta$)

Input: $\Pi_\mathsf{e}, \Pi_\mathsf{h}$ are two pure formulas, θ is a substitution.
Output: `Success` or `Fail`$\langle \Pi_\mathsf{u} \rangle$, where Π_u is the unmatchable pure formula.

```
if IsValid(Π_e → Π_h θ) then
    return Success
Π_1 ∧ ... ∧ Π_n ← CNF(Π_h)                    // transform Π_h to CNF
Π_u ← true
for each i in 1, ..., n do
    if ¬IsValid(Π_e → Π_i θ) and ¬IsValid(Π_u → Π_i) then
        Π_u ← Π_u ∧ Π_i
return Fail⟨Π_u⟩
```

Learning From Failure: The Proof Search Procedure

Procedure Prove($\mathcal{H}, F_1 \vdash F_2$)

Input: $\mathcal{H}$ is a set of induction hypotheses, $F_1 \vdash F_2$ is a goal entailment.
Output: The validity of the goal entailment $F_1 \vdash F_2$ (`Valid`, `Unkn`) or of an ancestor entailment $G_1 \vdash G_2$ (`ValidAncestor`$\langle \mathcal{H}', G_1 \vdash G_2 \rangle$).

```
1:  R ← FindRules(H, F1 ⊢ F2)
2:  for each R in R do
3:      if R ∈ {⊥_L^π, ⊥_L^σ, ⊢_π} then return Valid                    // axiom rule
4:      else if R ∈ {G_c, G_v, W_π} then                    // weakening/generalization rules
5:          G1 ⊢ G2 ← GoalEntailment(R)
6:          (H', G'1 ⊢ G'2) ← Premises(R)
7:          if Prove(H', G'1 ⊢ G'2) = Valid then
8:              return ValidAncestor⟨H', G1 ⊢ G2⟩
9:      else                                                        // other inference rules
10:         (H_i, F1i ⊢ F2i)_{i=1...n} ← Premises(R)
11:         (res_i)_{i=1...n} ← Prove(H_i, F1i ⊢ F2i)_{i=1...n}
12:         if ∀i=1...n. (res_i = Valid) then                       // all sub-goals are proved
13:             return Valid
14:         else if ∃i=1...n. (res_i = ValidAncestor⟨H', G1 ⊢ G2⟩) then
15:             if (F1 ⊢ F2) = (G1 ⊢ G2) then return Valid
16:             else return ValidAncestor⟨H', G1 ⊢ G2⟩              // backjump
17: return Unkn                                                     // all rules fail to prove F1 ⊢ F2
```

Learning From Failure: The Find Rule Procedure

Procedure FindRules($\mathcal{H}, F_1 \vdash F_2$)

$\mathcal{R} \leftarrow$ FindStandardRules($\mathcal{H}, F_1 \vdash F_2$) // *normal inference rules*
for each $\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$ **in** $\mathcal{H}$ **do** // *inference rules discovered by using IHs*
  if $F_1 \equiv \Sigma_1 \wedge \Pi_1$ **and** $\exists \mathsf{P}(\vec{t}) \in \Sigma_1$ **where** $\mathsf{P}(\vec{t}) \prec \mathsf{P}(\vec{r})$ **then**
    $\mathcal{U} \leftarrow$ MatchSpatial($\Sigma_3 * \mathsf{P}(\vec{r}), \Sigma_1, \varnothing$)
    $G_1 \vdash G_2 \leftarrow$ UnrenameIH($\Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4$) // *get the ancestor entailment*
    $\mathcal{H}' \leftarrow$ GetCorrespondingIHs($G_1 \vdash G_2, \mathcal{H}$) // *$\mathcal{H}'$ corresponds to $G_1 \vdash G_2$*
    for each $\mathtt{Success}\langle\theta, \Sigma\rangle$ **in** $\mathcal{U}$ **where** $\mathsf{P}(\vec{t})\theta \cong \mathsf{P}(\vec{r})$ **do**
      if MatchPure(Π_3, Π_1, θ) $= \mathtt{Fail}\langle\Pi_\mathsf{u}\rangle$ **then**
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleCA($\mathcal{H}, F_1 \vdash F_2, \Pi_\mathsf{u}\theta$)$\}$ // *the case-analysis rule* CA
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleW$_\pi$($\mathcal{H}', G_1 \vdash G_2, \Pi_\mathsf{u}$)$\}$ // *the weakening rule W_π*
      else
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleIH($\mathcal{H}, F_1 \vdash F_2, \Sigma_3 * \mathsf{P}(\vec{r}) \wedge \Pi_3 \vdash F_4, \theta, \Sigma$)$\}$ // *the rule* IH
    for each $\mathtt{Fail}\langle m\rangle$ **in** $\mathcal{U}$ **do**
      if HasOnlyOneNonMatchableConst(m) **then**
        $c, pos \leftarrow$ GetNonMatchableConstAndPosition(m)
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleG$_\mathsf{c}$($\mathcal{H}', G_1 \vdash G_2, c, pos$)$\}$ // *the generalization rule G_c*
      if HasOnlyOneNonMatchableVar(m) **then**
        $x \leftarrow$ GetNonMatchableVar(m)
        for each t **where** $\mathtt{Var}\langle x, _, t\rangle \in m$ **do**
          $\rho \leftarrow \{pos \mid \mathtt{Var}\langle x, pos, t\rangle \in m\}$
          $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleG$_\mathsf{v}$($\mathcal{H}', G_1 \vdash G_2, x, \rho$)$\}$ // *the generalization rule G_v*
return $\mathcal{R}$ // *all discovered rules*

Thesis Revision Plan

- Based on the comments of Prof. Joxan Jaffar:
  - Clarify and demonstrate the novelty of the mutual induction proof system.
  - Revise the comments on the related work [Chu et al., PLDI'15].

Thesis Revision Plan

- Based on the comments of Prof. Joxan Jaffar:
  - Clarify and demonstrate the novelty of the mutual induction proof system.
  - Revise the comments on the related work [Chu et al., PLDI'15].
- Based on the comments of Prof. David Rosenblum:
  - Update the specification of the motivating example in Chapter 1.
  - Reorganize the background of separation logic in Chapters 4, 5 and 6 into the same Chapter 2.
  - Update literature review on techniques handling more complicated data structures (DAGs, B-trees, AVL trees).
  - Revise the proof search heuristics, the name of learning from failure rules.
  - Explain challenges in future works: magic wand.

Thesis Revision Plan

- Based on the comments of Prof. Alwen Tiu:
  - Revise the semantics of inductive heap predicates.
  - Formally present the least fixed point semantics.
  - Revise the syntax of formulas, especially some syntactic sugars, to correspond with the syntax of the symbolic-heap separation logic fragment.
  - Clarify the soundness theorems and soundness proofs.
  - Reorganize the background of separation logic in Chapters 4, 5 and 6 into the same Chapter 2.
  - Use the term *matching* instead of *unification* in the IH application.
  - Revise the related works and other detailed issues related to the presentation.

References i

Berdine, J., Calcagno, C., and O'Hearn, P. W. (2004).
A Decidable Fragment of Separation Logic.
In *International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, pages 97–109.

Berdine, J., Calcagno, C., and O'Hearn, P. W. (2005).
Symbolic Execution with Separation Logic.
In *Asian Symposium on Programming Languages and Systems (APLAS)*, pages 52–68.

Bozga, M., Iosif, R., and Perarnau, S. (2008).
Quantitative Separation Logic and Programs with Lists.
In *International Joint Conference on Automated Reasoning (IJCAR)*, pages 34–49.

Brotherston, J., Distefano, D., and Petersen, R. L. (2011).
Automated Cyclic Entailment Proofs in Separation Logic.
In *International Conference on Automated Deduction (CADE)*, pages 131–146.

References ii

Brotherston, J., Fuhs, C., Pérez, J. A. N., and Gorogiannis, N. (2014).
A decision procedure for satisfiability in Separation Logic with inductive predicates.
In *Joint Meeting of International Conference on Computer Science Logic and Symposium on Logic in Computer Science, CSL-LICS*, pages 25:1–25:10.

Bundy, A. (2001).
The automation of proof by mathematical induction.
In *Handbook of Automated Reasoning (in 2 volumes)*, pages 845–911.

Chin, W.-N., David, C., Nguyen, H. H., and Qin, S. (2012).
Automated verification of shape, size and bag properties via user-defined predicates in Separation Logic.
Science of Computer Programming (SCP), 77(9):1006–1036.

Chu, D.-H., Jaffar, J., and Trinh, M.-T. (2015).
Automatic induction proofs of data-structures in imperative programs.
In *Conference on Programming Language Design and Implementation (PLDI)*, pages 457–466.

References iii

Cook, B., Haase, C., Ouaknine, J., Parkinson, M. J., and Worrell, J. (2011).
Tractable Reasoning in a Fragment of Separation Logic.
In *International Conference on Concurrency Theory (CONCUR)*, pages 235–249.

Enea, C., Lengál, O., Sighireanu, M., and Vojnar, T. (2014).
Compositional Entailment Checking for a Fragment of Separation Logic.
In *Asian Symposium on Programming Languages and Systems (APLAS)*, pages 314–333.

Enea, C., Sighireanu, M., and Wu, Z. (2015).
On automated lemma generation for separation logic with inductive definitions.
In *International Symposium on Automated Technology for Verification and Analysis (ATVA)*, pages 80–96.

References iv

Gu, X., Chen, T., and Wu, Z. (2016).
A complete decision procedure for linearly compositional separation logic with data constraints.
In *Automated Reasoning - 8th International Joint Conference, IJCAR 2016, Coimbra, Portugal, June 27 - July 2, 2016, Proceedings*, pages 532–549.

Hou, Z., Clouston, R., Goré, R., and Tiu, A. (2014).
Proof search for propositional abstract separation logics via labelled sequents.
In *Symposium on Principles of Programming Languages (POPL)*, pages 465–476.

Hou, Z., Goré, R., and Tiu, A. (2015).
Automated theorem proving for assertions in separation logic with all connectives.
In *International Conference on Automated Deduction (CADE)*, pages 501–516.

Iosif, R., Rogalewicz, A., and Simácek, J. (2013).
The Tree Width of Separation Logic with Recursive Definitions.
In *International Conference on Automated Deduction (CADE)*, pages 21–38.

References v

Iosif, R., Rogalewicz, A., and Vojnar, T. (2014).
Deciding Entailments in Inductive Separation Logic with Tree Automata.
In *International Symposium on Automated Technology for Verification and Analysis (ATVA)*, pages 1–218.

Lee, W. and Park, S. (2014).
A proof system for separation logic with magic wand.
In *Symposium on Principles of Programming Languages (POPL)*, pages 477–490.

Nguyen, H. H. and Chin, W.-N. (2008).
Enhancing Program Verification with Lemmas.
In *International Conference on Computer Aided Verification (CAV)*, pages 355–369.

Nguyen, H. H., David, C., Qin, S., and Chin, W.-N. (2007).
Automated verification of shape and size properties via Separation Logic.
In *Int. Conf. on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pages 251–266.

References vi

Pérez, J. A. N. and Rybalchenko, A. (2011).
Separation Logic + Superposition Calculus = Heap Theorem Prover.
In *Conference on Programming Language Design and Implementation (PLDI)*, pages 556–566.

Pérez, J. A. N. and Rybalchenko, A. (2013).
Separation Logic Modulo Theories.
In *Asian Symposium on Programming Languages and Systems (APLAS)*, pages 90–106.

Piskac, R., Wies, T., and Zufferey, D. (2013).
Automating Separation Logic Using SMT.
In *International Conference on Computer Aided Verification (CAV)*, pages 773–789.

Piskac, R., Wies, T., and Zufferey, D. (2014).
Automating Separation Logic with Trees and Data.
In *International Conference on Computer Aided Verification (CAV)*, pages 711–728.

References vii

Qiu, X., Garg, P., Stefanescu, A., and Madhusudan, P. (2013).
Natural proofs for structure, data, and separation.
In *Conference on Programming Language Design and Implementation (PLDI)*, pages 231–242.

Schrijver, A. (1986).
Theory of Linear and Integer Programming.
John Wiley & Sons, Inc., New York, NY, USA.

Ta, Q., Le, T. C., Khoo, S., and Chin, W. (2016).
Automated mutual explicit induction proof in separation logic.
In *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, pages 659–676.

Ta, Q., Le, T. C., Khoo, S., and Chin, W. (2018a).
Automated Induction Proof in Separation Logic: Learning from Failure.
In *Technical Report (TR)*, pages 1–34.

References viii

Ta, Q., Le, T. C., Khoo, S., and Chin, W. (2018b).
Automated Lemma Synthesis in Symbolic-Heap Separation Logic.
In *Symposium on Principles of Programming Languages (POPL)*, pages 9:1–9:29.

Xu, Z., Chen, T., and Wu, Z. (2017).
Satisfiability of compositional separation logic with tree predicates and data constraints.
In *International Conference on Automated Deduction (CADE)*, pages 509–527.